

1 **The `samplr` package: A tool for modeling human cognition with sampling**
2 **algorithms**

3 Lucas Castillo¹, Yun-Xiao Li¹, and Adam N. Sanborn¹

4 ¹University of Warwick, UK

5 **Author Note**

6 Corresponding Author: Lucas Castillo (castillo.lucas@protonmail.com)

Abstract

Many models of cognition have sampling as a central component. These have been used to explain human performance across a wide range of tasks, involving perception, probability reasoning, and choice. One family of these models takes inspiration from sampling approaches developed in statistics, either by adjusting samples with prior beliefs or by implementing a sampling algorithm that overcomes the difficulty of independent samples at the expense of autocorrelation. While these models have been successful in modeling human behavior, their adoption has been slowed by the technical difficulties in applying them. Here, we introduce the `samplr` package, which includes a variety of these algorithms and cognitive models built upon them. We show how models in this package can match qualitative patterns found in human data, and walk readers through examples of how to compare these models quantitatively to human data. This tool aims to make these methods accessible to many more researchers.

Keywords: sampling, R package, MCMC, Bayesian

The **samlr** package: A tool for modeling human cognition with sampling algorithms

Introduction

Computational models are precise formalizations of theories that organize and condense research findings and can make predictions for novel experimental designs. In studies of cognition, many computational models rely on the computational motif of *sampling*. For example, sequential sampling models (e.g., the Drift Diffusion Model, Linear Ballistic Accumulator, and Exemplar-Based Random Walk), assume that sampling of percepts or memories drives an internal accumulation process that maps to human choices and response times (Brown & Heathcote, 2008; Nosofsky & Palmeri, 1997; Ratcliff & McKoon, 2008). These sequential sampling models have been augmented to explain not only choices but also metacognitive judgments such as choice confidence (e.g., Hellmann et al., 2023; Pleskac & Busemeyer, 2010).

Other kinds of sampling models have been used to explain continuous estimates (Smith, 2016) and there has been work to integrate choices, estimates and response times (Kvam et al., 2023). Separate models have used sampling to explain human-generated confidence intervals (Juslin et al., 2007). Finally, another class of sampling models have been used to explain people’s probability judgments, one that assumes that the fallacies that people show in this task are caused by inaccurate counting of samples (Costello & Watts, 2014).

Computer scientists and statisticians also use sampling algorithms in their work, but for a practical purpose: to efficiently approximate complex probability distributions that otherwise would be intractable. These sampling algorithms use various tricks to provide efficient estimates. One trick, Bayesian Monte Carlo, improves the accuracy of estimates through the use of prior information (Rasmussen & Ghahramani, 2003). For example, when estimating the probability of an event through sampling, a prior distribution for the estimates is selected, and this prior distribution is updated by the

samples generated by the sampling algorithm. This process improves the estimate based on a finite number of samples – in the extreme case of a single sample it moderates the resulting estimate away from 0 or 1 – and so can save time that otherwise would have been spent generating a larger number of samples. This trick improves efficiency, but as a consequence it can cause probability estimates to disagree with one another, producing the same probabilistic reasoning fallacies that people show in this task (Zhu et al., 2020).

A second and more commonly used trick for improving the efficiency of sampling is to change the nature of the algorithm itself. Sampling directly and independently from the probabilities, as most cognitive models of sampling assume, requires knowing the probability of each and every event. This is feasible when the form of the distribution is known (e.g., Gaussian) or the number of events is small, but outside of these restricted situations it is much more difficult. For example, in a belief updating task, one may know the prior probability and the likelihood of an event, and the first step of calculating the posterior distribution of interest is to multiply the prior and likelihood together. This is easy, but the next step, dividing by the normalizing constant (i.e., the sum of each prior and likelihood across all possible events), which is needed to ensure that the distribution sums to one, can be intractably difficult. Computer scientists and statisticians have thus developed a family of algorithms, Markov Chain Monte Carlo (MCMC; Metropolis et al., 1953; Neal, 1993; Van Ravenzwaaij et al., 2018), that avoid the need to calculate the normalizing constant by only requiring ratios of probabilities to run – here the normalizing constants will cancel, and are not needed. The cost of these algorithms however is to introduce autocorrelation, therefore making the collection of samples less informative than an independent set of samples would be, as well as a dependence on the starting point. Such dependencies are like those found in repeated judgments of people (Zhu et al., 2022) and in the anchoring effect (Lieder et al., 2018).

Because statistical sampling algorithms were developed as general-purpose and efficient approximations, and because they are congruent with the motif of sampling found

in models of human behavior, they make for interesting candidates for modeling this behavior. However, despite this approach having been proposed more than two decades ago (Hoyer & Hyvärinen, 2002) and its successful application across a range of tasks, it only intermittently appears in the literature.

We believe one of the reasons for such sparseness is the difficulty in applying sampling algorithms to explain human behavior. Therefore, taking inspiration from the success of software packages implementing sequential sampling algorithms (Singmann et al., 2018; Vandekerckhove & Tuerlinckx, 2008; Wiecki et al., 2013), here we offer an R package, available in the CRAN repository, that implements the different statistical sampling algorithms. Our goals are to increase the coherence of separate investigations and to increase their efficiency. We accomplish both of these goals by providing a code base, the **samplr** package, in which many of the functions are implemented in C++ for speed.

The **samplr** package (see Figure 1) includes a variety of MCMC sampling algorithms—such as Metropolis-Hastings (MH), Hamiltonian Monte Carlo (HMC), and their variants—as well as cognitive models like the Bayesian Sampler (Zhu et al., 2020) and the Autocorrelated Bayesian Sampler (ABS; Zhu et al., 2024). Because of their implementation in a low-level programming language, these run up to four times faster than they would if coded in MATLAB, and much faster than if coded in base R (see package vignette). The package also offers diagnostic functions for analyzing autocorrelation, power spectral density, and other features that have been used to compare sampling algorithms to human behavior, which can be used in conjunction to more domain-specific diagnostic tools experimenters might employ.

In the following sections, we first demonstrate how to use the sampling algorithms and cognitive models in the **samplr** package to reproduce qualitative patterns observed in empirical studies. We then show how the **samplr** package can be integrated with likelihood-free inference tools to enable quantitative model comparisons at the level of individual participants (in one of the tools we show, we use the external **abcrf** package).

Throughout, we use version 1.1.2 of the package and provide short examples of code with example outputs. Interested readers may also want to examine the package documentation (<https://lucas-castillo.github.io/samplr/>), the supplementary materials to this paper in which we show how our figures were produced (<https://github.com/lucas-castillo/samplr-paper>), and the Appendix, which contains implementation details.

Qualitative Patterns

First we review examples of sampling algorithms and cognitive models qualitatively matching patterns of human behavior, across response modes such as probability judgments, estimates, choices, response times, confidence intervals and confidence reports. We also present an example of sampling algorithms explaining how response modes can interact: how choices can bias subsequent estimates, sometimes attracting and sometimes repulsing them.

Probability Judgments

A large body of research has shown that people commit systematic errors when reasoning with probabilities. For example, their probability judgments are lower if the question includes unlikely examples, as opposed to likely examples (*implicit subadditivity effect*; Dasgupta et al., 2017). Sometimes, their judgments even violate Kolmogorov probability axioms: estimates of the probability of an event “A” may be smaller than those of the conjunction “A and B” (*conjunction fallacy*; Hertwig et al., 2008; Tversky & Kahneman, 1983); and estimates of “A” and of the converse and mutually exhaustive “not A” might add up to more than one (violation of *binary complementarity*; Huang et al., 2024; Macchi et al., 1999; Tee et al., 2024).

A useful approach to study probability judgments has been to query participants on the probabilities of two events “A” and “B”, their complements “not A” and “not B”, and their conjunctions and disjunctions (e.g., “A and not B”). These responses can then be added or subtracted to produce a set of identities (called Z identities; for example, the sum

of the estimates of “A” and of “B” should equal the sum of “A and B” and “A or B”), which helps characterize systematic errors in probability reasoning (Costello & Watts, 2014, 2016, 2018).

A model that explains people’s errors in probability judgments is Probability Theory plus Noise (PT+N; Costello & Watts, 2014, 2017). According to this model, to judge the probability of an event people sample instances of events from memory according to their true probability, but sometimes misread them (e.g. mistakenly count a sample “A” as “not A”) with a fixed probability d . Alternatively, according to the Bayesian Sampler model, samples are never misread, but instead are adjusted by a Beta prior on responses. Surprisingly, these two models make identical predictions with regards to the mean of probability judgments (Costello & Watts, 2019; Zhu et al., 2020). However, their predictions on the relationship between the mean judgments and their variance are different, due to PT+N adjusting samples as they are drawn, while the Bayesian Sampler adjusts the estimate after the tally (Sundh et al., 2023). In the Bayesian Sampler, a smaller number of samples will lead to the endpoints of the mean-variance curve approaching the center, while in PT+N the endpoints of the curve will not be affected (see Figure 2).

With the `samplr` package, users can compute this set of identities by using the `Z_identities` function (we provide abridged and rounded example outputs when relevant, preceded by “#>”), completing parameters with the person’s estimates of the probabilities of events `a` and `b`, as well as their conjunctions (e.g., `a_and_b`), disjunctions (e.g., `a_or_b`), and conditionals (e.g., `a_given_b`).

```
Z_identities(
  a=.5, b=.2,
  a_and_b = .4, a_or_b = .3,
  a_given_b = .5, b_given_a = .3
)
#> | z1 | z2 | ... | z9 | ... | z15 | z16 |
#> |----|----|...|----|...|----|----|
#> |  0 | NA | ...|-0.05|...| 0.3 | 0.25|
```

Researchers can also use the `Bayesian_Sampler` function to simulate and make predictions with the Bayesian Sampler model; by inputting the true probabilities, the symmetric Beta prior (`beta`) and fixed sample size (`N`) parameters of the model, and whether the function should return the expected mean, expected variance, or a simulation of the outcomes:

```
Bayesian_Sampler(
  a_and_b = 1/8, b_and_not_a = 2/8,
  a_and_not_b = 3/8, not_a_and_not_b = 2/8,
  beta = .4, N = 5,
  return = c("mean", "variance", "simulation")[1]
)
#> $a_and_b
#> [1] 0.18
#> $b_and_not_a
#> [1] 0.28
#> $a_and_not_b
#> [1] 0.39
```

Estimates

A surprising fact of human cognition is people's responses on seemingly unrelated trials can actually be predictive of one another. In Gilden et al. (1995), participants listened to an auditory stimulus and then were asked to repeatedly reproduce it, matching its duration. They found that the errors participants made were not independent of each other, but instead depended on previous errors, both recent and old. Expressed as a power law relationship ($S(f) \sim 1/f^\alpha$) between the spectral power ($S(f)$) and the frequency (f), they found that the best-fitting exponent lay between white noise ($\alpha = 0$; independent errors) and Brownian noise ($\alpha = 2$; errors dependent on previous estimate only). This in-between region, called pink or $1/f$ noise ($\alpha \in [0.5, 1.5]$), represents long-range autocorrelations between estimates (Gardner, 1978).

Within a sampling framework, these temporal estimates can be characterized as samples drawn without aggregation into an inferential decision. Zhu et al. (2022) replicated Gilden et al.'s (1995) result, and used the resulting sequences to compare different

sampling algorithms that might show the same structure in estimate errors. They compared three different sampling algorithms, Metropolis-Hastings (MH; Hastings, 1970), Metropolis-coupled MCMC (MC³; Geyer, 1991) and Hamiltonian Monte Carlo (HMC; Neal, 2011), in their ability to produce long range autocorrelations in their estimates. Unlike the samples produced by the Bayesian Sampler above, these algorithms produce autocorrelated samples. Intuitively, MH iteratively produces a chain of samples by adding Brownian noise to the previous estimate, with an accept/reject stage that probabilistically discards a proposed sample if its density in the target distribution is smaller than the previous estimate. MC³ and HMC modify MH in different ways: MC³ runs several chains which occasionally swap positions, allowing for a chain to show large differences in consecutive estimates after a swap. HMC proposes new items in a more sophisticated way than adding noise (by simulating a physical system on the target distribution), and is thus less likely to reject proposals. Zhu et al. found that the algorithm running multiple chains (MC³) performed best and was able to reproduce the long-range autocorrelations found in human estimates (see Figure 3).

Another task where sampling algorithms have been used to understand human performance is the random generation task (Castillo, León-Villagr , et al., 2024; Li et al., 2024). In this task, participants are given a domain of interest (e.g. numbers one to ten; human heights) and they have to produce a sequence of items from this domain as if drawn “out of a hat” with replacement, i.e. completely independent of each other. A large body of research has shown that people’s sequences deviate from true random ones in systematic ways (Baddeley, 1966; Towse, 1998; Wagenaar, 1972). For example, the distance between items in people’s sequences is too small, and people almost never repeat the same item twice in a row.

In two experiments, Castillo, León-Villagr , et al. (2024) compared six sampling algorithms and a previous random generation model (Cooper, 2016) against human performance. Participants produced random heights, which in one condition were

distributed as in the general population (*Gaussian* condition) whilst in the other they were all equally likely (*Uniform* condition). The authors focused on the models' ability to replicate four patterns of sequential dependence found in human data as well as people's ability to match non-uniform distributions (see Table 1). They found that the best fitting model was MCREC, a sampling algorithm combining the proposal mechanism of HMC with the multiple chains idea of MC³ and with a sense of momentum, which adds autocorrelation to the direction of proposals (we compare data to prior predictives of this model in Figure 4).

These six MCMC sampling algorithms are available in the **samplr** package, in functions starting with **sampler_***. Users must indicate the target distribution and a starting point, as well as sampler-specific parameters. For example, the following code generates samples from a Standard Normal distribution using the classic Metropolis-Hastings algorithm:

```
sampler_mh(
  # Parameters all samplers require
  start = 0, distr_name = "norm", distr_params = c(0,1),
  # MH specific parameters
  sigma_prop = .5
)
#>$Samples
#>      [,1]
#> [1,]  0.00
#> [2,] -0.21
#> [3,] -0.62
#> [4,] -0.61
#> ...
```

These functions can sample from mixture distributions

```
sampler_mh(
  start = 0, distr_name = c("norm","norm"),
  distr_params = list(c(-1,1), c(1,1)),
  weights = c(.5, .5), # optional mixture weights parameter
  sigma_prop = .5, # MH specific parameter
)
```

as well as bespoke distributions:

```
custom_pdf <- function(x){
  if (x[1] < 4 & x[1] > 2){
    if (x[2] < 6 & x[2] > 4){
      return(1)
    }
  }
  return(0)
}
sampler_mh(
  start = c(3,3), custom_density = custom_pdf,
  sigma_prop = diag(2) * .5
)
```

Case Study – Sequentially estimating a latent variable

In this section, we use research by Bruckner et al. (2025) to show how the sampling algorithms available in the `samlr` package have been used in modeling cognition by other research groups. We use this case study to demonstrate how these modeling tasks can be approached using the `samlr` package for increased ease.

Bruckner et al. (2025) were interested in studying how people navigate situations in which the variability in sequential data might be due to typical fluctuations from a constant latent state, or to a change in the latent state altogether (e.g., a subpar meal at an often-visited restaurant being due to transitory bad luck or a permanent change of cook). In learning tasks where people repeatedly estimate the true latent mean of the generating process from data, people exhibit two prominent biases: perseveration, so that responses are identical to those in the previous trial despite new information having been received; and anchoring, so that responses are biased by external, irrelevant cues. These biases are more prominent in children and older adults, whose cognitive resources are more limited on average and so are more likely to perseverate (Cepeda & Munakata, 2007; Ridderinkhof et al., 2002) and to anchor their responses (Munakata et al., 2012).

Bruckner et al. (2025) developed a model incorporating MCMC algorithms to explain how these biases arise and why their magnitude changes across the lifespan. In

their task, participants needed to place a bucket along a horizontal strip with 300 possible locations, so that it would catch reward bags dropped by a helicopter whose position was occluded by clouds. The drop point was determined by a draw from a Gaussian distribution with mean the helicopter position and fixed non-zero variance. In some trials, however, the helicopter changed positions to any point in the strip. Therefore, a bag drop in a given location could be due to typical fluctuations or to a change in the latent helicopter position.

According to their computational model, when people estimate the true location of the helicopter in a trial they do so by producing samples from the posterior using the Metropolis-Hastings algorithm (implemented as `sampler_mh()` in the `samlr` package), in chunks so that resource demands of the task are manageable. The posterior is a weighted sum of the posterior beliefs conditional on a change point having or not occurred, with a free parameter determining the sum weights (i.e., the participant's a priori belief about the probability of a change point). Participants sample a chunk of n samples (a free parameter), then compare how different the mean of the samples is at predicting the data compared to the belief before sampling. If not very different (the tolerance for this is determined by the stopping criterion c , a free parameter), sampling stops and the initial belief is used as a response (samples are discarded). Otherwise, the belief is updated to the mean of the samples and a new chunk of samples are drawn. Once a response is given, the prediction error is used to recompute the width of the prior. Changes in perseveration and anchoring across the lifespan can be explained by larger n values when cognitive resources are high.

If they had used the `samlr` package, their model could have been implemented using the `sampler_mh()` function to produce the samples (on our machine, this runs 31 times faster than their code). Their code for a trial update might have looked as follows (the functions `calculate_posterior()`, `check_criterion()`, and `update_posterior_width()` carry out processes we verbally describe above, but we do not define them for brevity):

```

bruckner_et_al <- function(
  x, # observed bag drop
  current_belief, # current helicopter position belief
  h, # change point probability belief
  n, # samples per chunk
  c # stopping criterion
){
  error <- x - current_belief
  continue_sampling <- TRUE
  while(continue_sampling){
    posterior_params <- calculate_posterior(current_belief, x)
    samples <- sampler_mh(
      distr_name = c("norm", "unif"),
      distr_params = list(posterior_params, c(0,300)),
      weights = c(1-h, h),
      iterations = n
    )$Samples
    continue_sampling <- check_criterion(samples, c, h)
    if (continue_sampling){
      current_belief <- mean(samples)
    }
  }
  update_posterior_width(samples, error)
}

```

Of course, we have omitted some details here, as our objective was to show how `samplr` package functions can be used in a novel modeling task. Among details that we do not report here, we highlight that the model developed by Bruckner et al. (2025) can account for different learning rates depending on trial accuracy being more or less rewarded, for uncertainty being underestimated, for learning rates being affected by change-point probabilities, and for larger (or total) belief updating on catch trials where the helicopter is actually visible. The sampling component of the model, the Metropolis-Hastings algorithm, is identical in their implementation and the `samplr` package.

Choice Accuracy and Response Times

The speed-accuracy trade-off is one of the key relationships between accuracy and response time in psychological tasks. Extensive research has demonstrated that individuals

can prioritize accuracy by taking more time, or instead favor speed at the expense of accuracy (for a review, see Wickelgren, 1977). For instance, when accuracy is emphasized through instructions or incentives, individuals typically take longer to make judgments. Conversely, when speed is prioritized or decision time is constrained, judgments are often less accurate.

Within the framework of sequential sampling models that were developed in psychology, Ratcliff (1978) attributed this effect to the manipulation of the amount of information required for a judgment. More recent analysis using the Drift Diffusion Model showed that this trade-off arises primarily from variations in the boundary separation parameter, followed by changes in the non-decision time and drift rates (Ratcliff & McKoon, 2023).

The Autocorrelated Bayesian Sampler (ABS; Zhu et al., 2024), a cognitive model in the `samlr` package, imports this successful explanation into a statistical sampling framework. As illustrated in Figure 5, ABS is a sequential sampling model assuming people draw discrete and autocorrelated samples from a distribution, which represents signal strength, to make judgments and decisions. For two-alternative forced-choice tasks, ABS employs a relative stopping rule: sampling continues until the accumulated difference in evidence between the two alternatives exceeds a decision threshold.

Within this framework, the speed-accuracy trade-off arises from variations in the threshold level required by the task. Higher thresholds demand more evidence, resulting in longer response times but greater accuracy; lower thresholds require less evidence, producing faster but less accurate responses.

The following code demonstrates how to run ABS simulations to reveal this effect. ABS models were programmed using the R6 object-oriented programming system (Chang, 2025). Therefore, the first step in the simulation is to initialize the model using `$new()`, where users specify how the internal sampler operates, how samples are mapped to response time, and what the distribution to sample from is (`distr_name` specifies the name

of the distribution and `distr_params` provides its parameter values). Currently, ABS uses MC³ as its sampler, a Markov Chain Monte Carlo algorithm that generates new samples by adding normally distributed, unbiased noise to the previous item (`width` defines the width of the noise distribution). It runs multiple replicas, which swap positions stochastically (`n_chains` sets the number of chains; see more details in the Estimates section below, and in the Appendix). Response time is calculated as the sum of non-decision time (drawn from a uniform distribution $U[\text{nd_time}, \text{nd_time} + \text{s_nd_time}]$) and decision time (a random draw from an Erlang distribution $\text{Erlang}(k, \lambda)$, where k is the number of samples drawn by the sampler and λ is provided by the user [`lambda`]).

After the initialization, users can use the `$simulate()` function to run the simulation. For this example, where a relative stopping rule is used, users need to provide information about how different the stimuli are (`discrim`), and where the boundary dividing support for one or the other option lies (`dec_bdry`). Finally, the key parameter here is the evidence threshold (`delta`): if large (e.g., `delta = 10`) sampling will be longer and more accurate. Conversely, when speed is prioritized, a smaller threshold is sufficient (e.g., `delta = 3`), resulting in shorter sampling durations. The results of the simulation can be accessed via `$sim_results`.

```
stimuli <- factor(sample(c('left', 'right'),
                        size = 200, replace = TRUE))
ABS_1 <- Zhu23ABS$new(
  # sampler parameters --
  width = 0.01, n_chains = 8,
  # decision time parameters --
  nd_time = 0.4, s_nd_time = 0.3, lambda = 100,
  # posterior distr. parameters --
  distr_name = 'norm', distr_params = 1
)
ABS_1$simulate(
  stopping_rule = 'relative',
  delta = c(3, 10)[1], # prioritize speed or accuracy
  dec_bdry = 0,
  discrim = 1,
  trial_stim = stimuli
)
```

```

ABS_1$sim_results
#> | trial |      samples | response | stimulus | acc. | rt | ... |
#> |-----|-----|-----|-----|-----|-----| ... |
#>1|      1 | -1.2, -1.2, ... | left      | left      |      1 | 0.47 | ... |
#>2|      2 | -1.3, -1.1      | left      | left      |      1 | 0.48 | ... |
#>...

```

Figure 6 illustrates ABS simulations of accuracy and mean response time using the `samplr` package, where only the evidence threshold varies while all other parameters remain fixed. These simulations show that higher accuracy is consistently associated with longer response times.

Metacognition: Confidence Intervals

Subjective belief can be elicited through single point estimates, but also by asking participants to provide a confidence interval with a pre-stated probability. For instance, participants might be asked to provide a 90% confidence interval for Finland’s population. Research indicates that people often exhibit overconfidence, producing over-precise estimates in both general knowledge and sensory discrimination tasks (Juslin et al., 2003, 2007; Moore & Healy, 2008). In other words, their reported confidence intervals are narrower than they should be.

ABS follows the approach of Juslin et al. (2007) and assumes that, when providing estimates, people draw samples from a distribution and terminate the process once a fixed number of samples has been collected. To produce these within an ABS framework (as opposed to via sampling algorithms directly, as explained above), the procedure is very similar to producing choices, with the exception that results are simulated using a “fixed” stopping rule, where they need to provide the number of samples wanted (`n_sample`):

```

stimuli <- rnorm(123)
ABS_2 <- Zhu23ABS$new(...) # as above
ABS_2$simulate(
  stopping_rule = 'fixed',
  n_sample=10, trial_stim = stimuli
)
ABS_2$sim_results

```



```
#> | trial | samples | stimulus | rt | point_est |
#> |-----|-----|-----|-----|-----|
#> 1|      1 |      ... |    -0.63 | 0.73 |    -0.87 |
#> 2|      2 |      ... |     0.18 | 0.63 |    -0.60 |
#> ...
```

Confidence intervals are then constructed based on the quantiles of the samples drawn. When the number of samples is limited, the distributional tails are typically underrepresented, leading to a true proportion that is lower than the subjective probability (Figure 7). Confidence intervals can be added to ABS outputs by calling the `confidence_interval()` function, with the pre-stated probability as the only parameter:

```
# add 70% confidence interval to results
ABS_2$confidence_interval(0.7)
ABS_2$sim_results
#> |...| point_est | conf_interval_l | conf_interval_u |
#> |...|-----|-----|-----|
#> 1|...|    -0.87 |          -0.96 |          -0.66 |
#> 2|...|    -0.60 |          -0.88 |          -0.54 |
#>...
```

Metacognition: Confidence in Decisions

Confidence can also be construed not as a range where an estimate may lie, but as the degree to which participants believe they made the correct choice. Research has consistently shown that average confidence ratings tend to be higher for correct responses than for incorrect ones, an effect often called the *resolution of confidence* (e.g., Ariely et al., 2000; Baranski & Petrusic, 1998; Vickers, 1979; Vickers & Packer, 1982). This confidence effect persists even when stimulus difficulty is held constant (Baranski & Petrusic, 1998) and when choice and confidence are elicited simultaneously from participants (Kiani et al., 2014; Ratcliff & Starns, 2009).

ABS assumes that individuals generate confidence estimates by calculating the proportion of samples drawn during choice that support each alternative. Because choices are made using a relative stopping rule where the support of samples for or against an

option is autocorrelated, incorrect choices often occur when the evidence remains relatively balanced for an extended period, resulting in a slow error. In such cases, the total number of samples (i.e., the denominator in the confidence calculation) tends to be larger for errors than for correct responses (see Equation 9 in Zhu et al., 2024; Figure 8). For example, imagine a decision-maker who is asked whether a presented cloud of dots has more than 25 (as in Figure 5). If their MCMC sampler samples around the 25 dot boundary, with several samples above and below 25 representing support and opposition for the ‘yes’ response respectively, then their forced choice accumulation process will zigzag, taking a long time to reach one of the boundaries, and the proportion of samples for or against their choice will be closer to 50%. If on the other hand their MCMC sampler samples around 30, far from the boundary, successive samples will all be above 25, all supporting the ‘yes’ response, which will lead to a fast choice and high confidence.

In the `samlr` package, the confidence value for each choice is automatically stored in the `confidence` column of the `$sim_results` field. For the choices made above:

```
ABS_1$sim_results
#> | trial | samples | response | stimulus | acc. | rt | confidence |
#> |-----|-----|-----|-----|-----|-----|-----|
#>1|      1 |     ... | left     | left     |    1 | 0.47 |         0.80 |
#>2|      2 |     ... | left     | left     |    1 | 0.48 |         0.80 |
#>...
```

Choices Influence Subsequent Estimates: Anchoring and Repulsion Effects

Human response modes are not independent of one another – instead they are known to interact. In cognitive judgments, estimates are often biased towards an initial value that is part of a choice, even when that value is irrelevant to the task (Epley & Gilovich, 2006; Tversky & Kahneman, 1974). This well-documented phenomenon, known as the *anchoring bias*, reflects the tendency for initial values to influence subsequent judgments (this is also seen in the Bruckner et al. (2025) example shown above). However, more recent research in perceptual judgments, such as dot orientation or motion direction, has revealed a contrasting phenomenon: the *repulsion effect*, where estimates deviate away

from the initial value (Jazayeri & Movshon, 2007; Luu & Stocker, 2018; Zamboni et al., 2016).

Spicer et al. (2022) demonstrated that these seemingly opposed effects actually depend on the position of the initial value relative to the distribution of stimuli, and were similar across perceptual and cognitive stimuli. Specifically, estimates of the stimulus value are drawn toward the initial value if it is distant from the true value of the stimulus, replicating the anchoring effect, but are pushed away from the initial value if it is closer to the true value, replicating the repulsion effect.

These effects can be explained by the combination of an autocorrelated MCMC sampling algorithm that starts at the initial value and a relative stopping rule, two core components of ABS. In our simulation, we began with a set of distributions to represent the underlying stimulus distribution, illustrated by the black curves in Figure 9. To run the ABS model on custom distributions, users should specify the `custom_distr` and `custom_start` arguments when initializing `Zhu23ABS$new()`. These allow the user to define both the custom stimulus distributions and the corresponding starting values for the sampling process.

```
# create a list of density functions
M <- lapply(
  # for each possible mean...
  seq(-0.5, 0.5, 0.25),
  # ... make a density function
  \(mu){return(\(x){dnorm(x, mu)}})
)
# density function per trial
custom_dens_list <- rep(M, each=1000)

ABS_3 <- Zhu23ABS$new(width = 0.05, n_chains = 8, nd_time = 0.4,
                      s_nd_time = 0.3, lambda = 100,
                      custom_distr = custom_dens_list,
                      custom_start = 0)
```

When the starting point and the decision boundary (indicated by the green line) are close to the stimulus distribution and effectively partition the distribution into two binary

response regions, the relative stopping rule requires that the sampling process continue until a sufficient number of samples fall on one side of the boundary over the other. As a result, the final sample in the sequence, used as the estimate, tends to be away from the decision boundary, producing the repulsion effect. It is important to note that both fixed and relative stopping rules can yield point estimates, stored in the `point_est` column of the `$sim_results` field (when we showed outputs of the `ABS_1` model above, we omitted this column for clarity).

On the other hand, if the decision boundary and the starting point (indicated by the red line) are distant from the stimuli distributions, the sampling process is driven by two opposing forces. While the process is drawn toward the stimulus distribution, the autocorrelation inherent in the sampling process acts as a counterbalancing force. Because nearly all samples fall on the same side of the decision boundary, the process tends to terminate before fully exploring the stimulus distribution, resulting in the anchoring effect.

Finally, one might also notice that the green histogram in Figure 9 covers a narrower range than the full stimulus distribution (depicted by the black curves). This is because the sampling chains are autocorrelated and all begin at the green line, causing the final samples to cluster near the starting point, thus limiting their spread (i.e., anchoring is also occurring).

Quantitative Comparisons

So far, we have shown that the models here presented can qualitatively match patterns in human behavior. Researchers, however, need quantitative ways to compare models, and unfortunately, the models here presented are not tractable: no analytical solution exists for their likelihood function, and the computational cost of calculating it is too high. For this reason, we rely on likelihood-free approximations (for a recent review, see Cranmer et al., 2020, and see Palestro et al., 2018 and Sisson et al., 2019 for more depth).

Likelihood-free inference is general in nature, and can in principle be used on any inference problem. All that is needed is that candidate models can simulate data, and all

models in the `samplr` package can. For Approximate Bayesian Computation, which is the technique we will use below, researchers will also want a way to transform the data into a set of summaries. The `samplr` package offers some diagnostic functions (mostly starting with `calc_*`). For example, a user can run

```
calc_PSD(samples)
#>fit
#>slope intercept
#>-0.97 -1.20
calc_levy(samples)
#>coef
#>[1] -0.95 0.64
```

to obtain the power spectral density and Lévy flight coefficient of a sequence of samples. These measures have been used to characterize the long-range autocorrelations (Gilden et al., 1995) of estimates and the heavy-tailed distributions of response times (Rhodes & Turvey, 2007) found in human estimates, respectively (see also Zhu et al., 2022). Crucially, however, the choice of summary statistics is domain dependent, so depending on the specific application, the ones offered in the `samplr` package may need to be complemented by others.

Below, we give an example of how likelihood-free inference can be carried out, using Approximate Bayesian Computation (ABC). We perform quantitative analyses two ways: first, via rejection ABC (Beaumont et al., 2002), which will serve establish the reasoning behind how ABC works. Then, we will use a more recent version of Approximate Bayesian Computation, ABC with random forests (a machine learning algorithm; Turner and Van Zandt, 2012; Pudlo et al., 2016; see Hastie et al., 2009 for a general introduction to random forests).

Example: Quantitative Comparisons in Castillo, León-Villagr , et al. (2024)

In Experiment 1 of Castillo, Le n-Villagr , et al. (2024), participants generated a sequence of random heights (data from their experiments is available in the `samplrData` package; Castillo, Li, and Sanborn, 2024, which is also available on CRAN). They were

asked to imagine that every healthy adult in the UK had written their height on a piece of paper. In the *Gaussian* condition, all these pieces of paper had been put in a bag. In the *Uniform* condition, instead, the bag only contained unique values. Their task was to say one of the heights in the bag as if drawn, read, and restored each time. The authors compared participants' data to six local sampling algorithms (those in Figure 1) and another computational model of random generation (Cooper, 2016). For speed we simplify their analysis here to evaluating the *Gaussian* condition against only the sampling algorithms in the `samplr` package.

The objective of the authors was to determine which model $m \in \mathcal{M}$ best described a given datum y (i.e., a sequence of heights); with the hurdle that the likelihood of the data given a particular model is intractable. The logic of ABC is to approximate the posterior $\pi(\mathcal{M}|y)$ by simulating data from the prior over the candidate models. One draws a random model from the prior over models, $m' \sim \pi(\mathcal{M})$, then simulates data from said model, $m' \rightarrow y'_{m'}$. If one discards all synthetic data that are not identical to the observed datum, then the distribution of surviving generating models will approximate the true posterior eventually, by the law of large numbers. For example, trying to infer how many six-sided dice were rolled to produce a sum of 11, one could repeatedly roll n dice (with n drawn from a prior $n \sim [2, 10]$), then only tally the generating n if the roll added exactly to 11. This would, given enough rolls, approximate the correct posterior eventually.

In practice the requirement of models needing to produce synthetic data identical to the one observed is too strict, leading to prohibitively expensive inference as too many simulations are required. Instead, some of these requirements are relaxed, in an exercise balancing accuracy and computation (Sisson et al., 2019): First, summary statistics instead of full datasets are used, so that the posterior distribution $\pi(\mathcal{M}|y)$ is replaced by the partial posterior $\pi(\mathcal{M}|s_y)$ where $s_y = S(y)$ and $S(\cdot)$ is a function generating a vector of summary statistics from the data. If S is *sufficient* for $\mathcal{M}$, i.e. it captures all the information in the data that is relevant to the estimation of $\mathcal{M}$, then no loss of information

occurs at this stage (Didelot et al., 2011). Second, a tolerance ϵ is defined, so that synthetic data are only rejected if the distance between s'_y and s_y is greater than ϵ .

To return to the example of estimating which model best reproduces the sequences of random heights people produced, the authors first generated synthetic data with their models (here we produce 10,000 simulations per model). The sampling algorithms sampled from a normal distribution with mean $\mu = 176.4$ and standard deviation $\sigma = 12$ (derived from the data). The prior over models was uniform, and the prior over parameters is defined in Table 2, with simulation parameters being randomly drawn from this prior. The summary statistics used were those in Table 1. To reiterate, the choice of summary statistics when using this method is important as ideally they would be sufficient. However, this ideal is difficult to achieve in practice, partly because whether a set of summary statistics is sufficient is unknowable. Typically, the choice of summary statistics is based on experimenter knowledge only, although efforts to automate part of this process have been developed recently (Fearnhead & Prangle, 2012; Valentin et al., 2024).

Once the matrix of summary statistics and generating models is available, observed and synthetic summary statistics should be standardized (using the mean and SD from the synthetic data), so that all summaries are weighted equally. Following Biau et al. (2015), we use a convenience tolerance ϵ^* so that only 10% of simulations are kept, and we find that MCREC is the model with the largest support, with a $BF_{10} > 400$ over the next best model.

Whilst powerful, rejection ABC is not without limitations: it requires a considerable volume of simulations to approximate the posterior accurately, and it is sensitive to the choice of summary statistics. For example, the set we use here (Table 1) is not a set of orthogonal statistics, and it could be argued that some aspects of the sequence, like the distance between items, is evaluated by more than one summary statistic (like *Adjacencies* and *Distances*) whereas other aspects, like the tendency to change from a descending to an ascending run or vice versa, is only measured by one (*Turning Points*). Therefore, if all

summary statistics are weighted equally (as we do) then models that do not replicate the distances between items well will be penalized more strongly when computing the posterior.

Pudlo et al. (2016) developed an ABC technique that addresses these limitations. Instead of evaluating the matrix of summary statistics directly, they run it through a machine learning classifier, a random forest, that learns the non-linear relationships between summary statistics that best predict which model produced the synthetic data, therefore weighing each summary statistic as a function of its predictive value, and needing fewer simulations than rejection ABC. Once trained with the simulations, the observed data serve as the test data that random forests allocate to generating models.

For convenience, we use the `abcrf` package here (Pudlo et al., 2016) to perform inference on the Castillo, León-Villagr , et al. (2024) data. We reuse the simulations and summaries generated above, then train the random forest using the `abcrf()` function their package provides. Once the forest has been trained, a useful metric is the out-of-bag error: the percentage of trees that haven't seen a given simulation during training and misclassify it during this checking step. In our case, with the `abcrf` default settings, this is 28.27%.

The final step is to ask the random forest classifier to allocate models to the observed data. For each tree, the posterior distribution over models is the relative frequency of model labels in the training data for which the same classification was made. In other words, the tree will classify the data and then remember what the proportion of true generating models was, during training, when the same leaf in the tree was reached. The posterior distribution of the forest is the average of tree posteriors, which is accurate given sufficient trees. Using this approach, the support for MCREC was $BF_{10} > 10^3$ over the next best model. We show model classification results for each participant in Figure 10.

Parameter estimation follows a very similar procedure for rejection ABC: first, simulations are generated by drawing parameter vectors from the prior over parameters, and after rejecting all simulations for which the vector of summary statistics is too far from the data, the distribution of surviving parameters approximates the posterior over

parameters. For a treatment of parameter estimation with random forests, see Raynal et al., 2019 (also implemented in the **abcrf** package).

Discussion

We presented the **samlr** package, an R package available in the CRAN repository that contains several statistical sampling algorithms, as well as cognitive models using sampling, to compare to human data – in addition to many diagnostic tools to aid this process. We have shown how these cognitive explanations can match a wide range of interesting qualitative observations of human behavior – be it errors in probability judgments; relationships between choice and speed or confidence; starting point influences; or the characteristic long-range structure of repeated estimates. Also, we have presented two of the many ways that these models can be evaluated quantitatively, giving a short example of how to use rejection-based Approximate Bayesian Computation as well as how to use random forests for ABC as proposed by Pudlo et al. (2016).

We invite researchers studying different psychological and behavioral phenomena to try these approaches, and we provide support with the commented examples that supplement this paper, showing how our figures were produced. As a potential starting point, datasets collected by our lab – which have already been evaluated using these approaches – are available in the **samlrData** package (Castillo, Li, & Sanborn, 2024), also available on CRAN.

After all, an exciting feature of these statistical sampling algorithms is how general they are in purpose: they have been appealed as explanations of phenomena from low-level perception (Gershman et al., 2012) to causal reasoning (Bramley et al., 2017; Davis & Rehder, 2020). What is more, they are not limited to very simple representations as other cognitive models might be: for example, in an experiment involving descriptions of people, experimenters could transform the text participants read into a semantic vector space (e.g., as done by Bhatia, 2017), and then use statistical sampling algorithms to sample from that space with the probabilities given by a complex neural network (Sanborn et al., 2024).

A particularly exciting possibility is that potentially there is a single sampling approach that can explain a wide range of judgments and behaviors. The ABS uses a single sampling process and simple mapping rules to explain probability judgments, choices, response times, confidence judgments, estimates, and confidence intervals, and even interactions between these response modes (Zhu et al., 2024). Being able to quantitatively fit the same model to such a wide range of judgments will provide much greater insight into human perception and cognition than examining separate tasks and response modes in isolation, and potentially make much more powerful predictions.

We hope that by making this set of functions available, we can facilitate using these approaches as well as make their implementations homogeneous; akin to how software packages implementing sequential sampling algorithms have done so (Singmann et al., 2018; Vandekerckhove & Tuerlinckx, 2008; Wiecki et al., 2013). Importantly, the `samplr` package is free and open software in active development, and we welcome requests for new features as well as invite other researchers to expand the software.

Author Declarations

Funding. LC, YL and ANS received funding from the European Research Council (Grant agreement No. 817492-SAMPLING), awarded to ANS.

Conflicts of interest. The authors declare no conflicts of interest.

Ethics Approval. Not Applicable.

Consent to Participate. Not Applicable.

Consent for Publication. Not Applicable.

Availability of data and materials. Code used to produce the figures in this article is available at <https://github.com/lucas-castillo/samplr-paper>.

Code Availability. The software here presented is available in the CRAN repository (<https://cran.r-project.org/web/packages/samplr/>), and its development version is available at <https://github.com/lucas-castillo/samplr>.

Acknowledgments

570

571

We thank Rasmus Bruckner for useful feedback.

Functions in the **samplr** package

Sampling Algorithms

Metropolis-Hastings

`sampler_mh()`

MC³

`sampler_mc3()`

Hamiltonian Monte Carlo

`sampler_hmc()`

REC

`sampler_rec()`

MCHMC

`sampler_mchmc()`

MCREC

`sampler_mcrec()`

Cognitive Models

Bayesian Sampler

`Bayesian_Sampler()`

Autocorrelated

Bayesian Sampler

`Zhu23ABS()`

Diagnostic Functions

Autocorrelations

`calc_autocorr()`

Sigma Scaling

`calc_sigma_scaling()`

Power Spectral Density

`calc_PSD()`

QQ-Plots

`calc_qqplot()`

Lévy Flight

`calc_levy()`

Probabilistic (Z) Identities

`Z_identities()`

Figure 1

*Inventory of functions currently implemented in the **samplr** package. In addition to several statistical sampling algorithms whose behavior has previously been compared to human data, the package offers cognitive models, as well as a range of diagnostic functions which can be used when examining model performance quantitative or qualitatively. In grey, names of the functions in the package.*

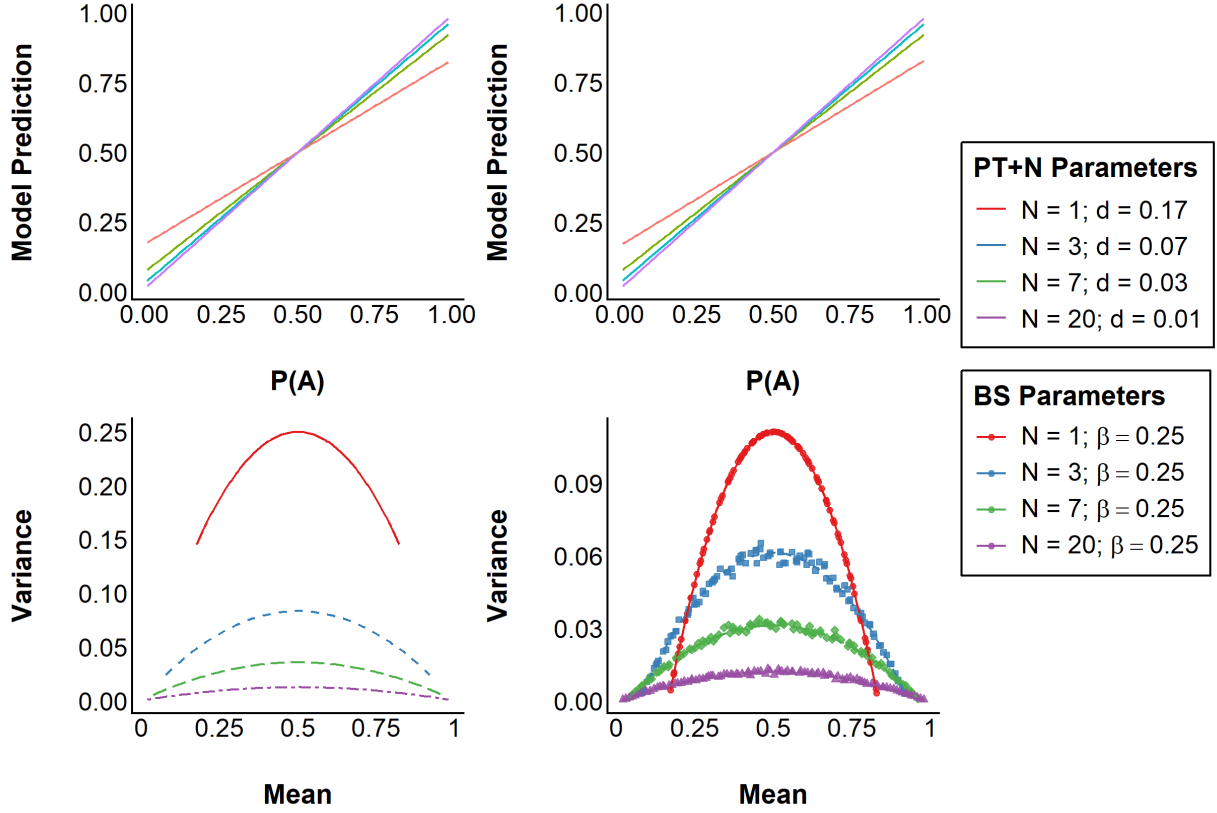


Figure 2

Results of the Bayesian Sampler (Zhu et al., 2020) and Probability Theory plus Noise (Costello & Watts, 2014). While model predictions are identical for mean estimates, they make different predictions regarding the mean-variance relationship (Sundh et al., 2023).

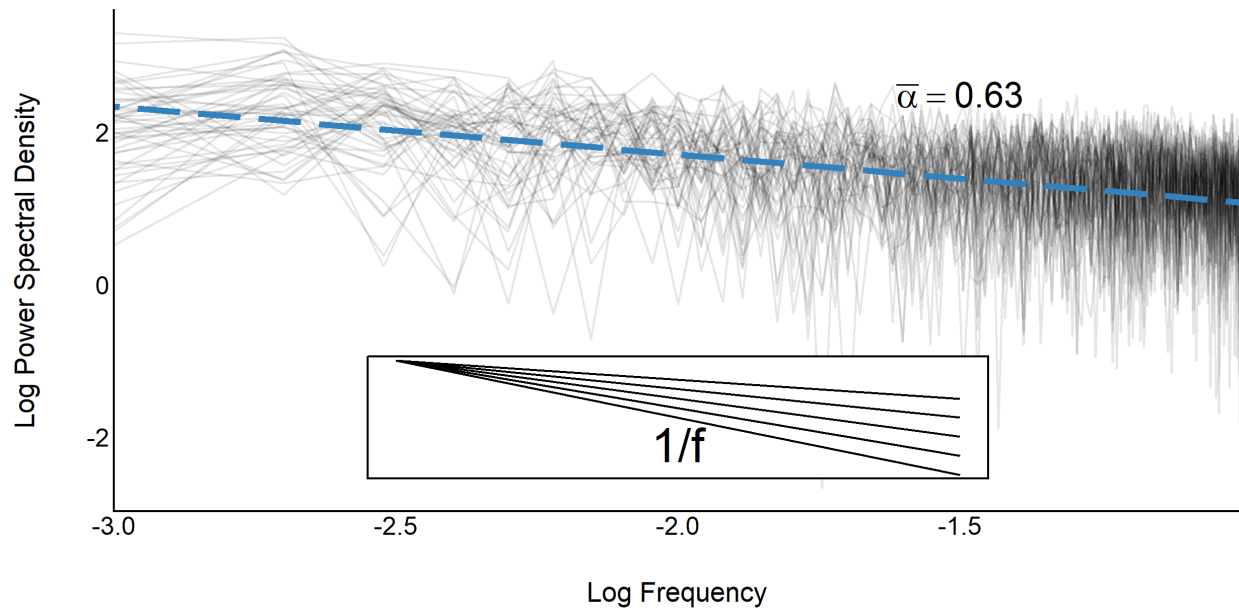


Figure 3

Power spectral analysis of 50 simulations of MC^3 , estimating a normal distribution. The relationship between the spectral power $S(f)$ and the frequency f is best described as a power law $S(f) \sim 1/f^\alpha$ with exponents $\alpha \in [0.5, 1.5]$, consistent with $1/f$ noise (for convenience we show this range in the inset).

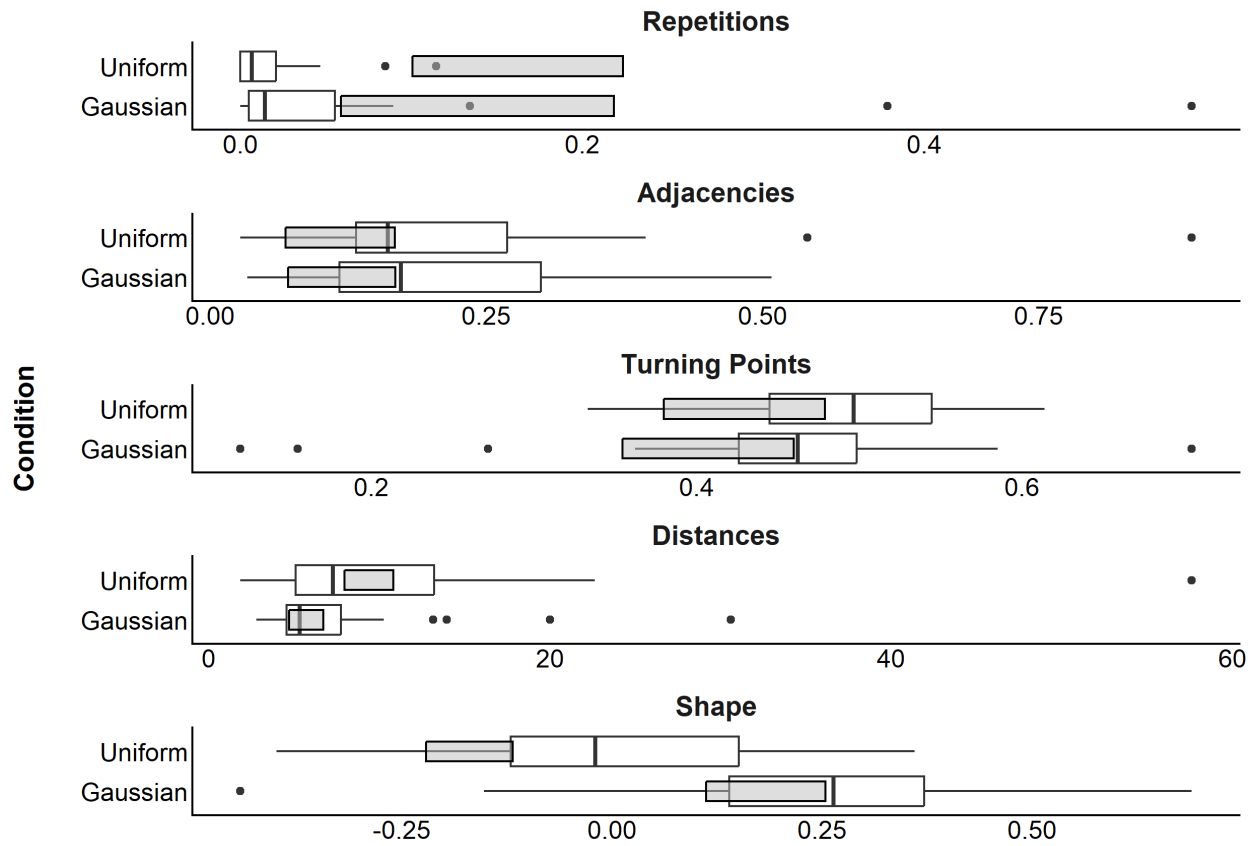


Figure 4

Random Generation Patterns (see Table 1). Boxplots show data from Experiment 1 in Castillo, León-Villagr , et al. (2024), where participants produced random heights, for the Uniform (all heights equally likely) and Gaussian (heights distributed as in the general population) conditions. Grey rectangles show the mean values of MCREC simulations $\pm$ 0.5 standard deviations, using parameters drawn from their prior.

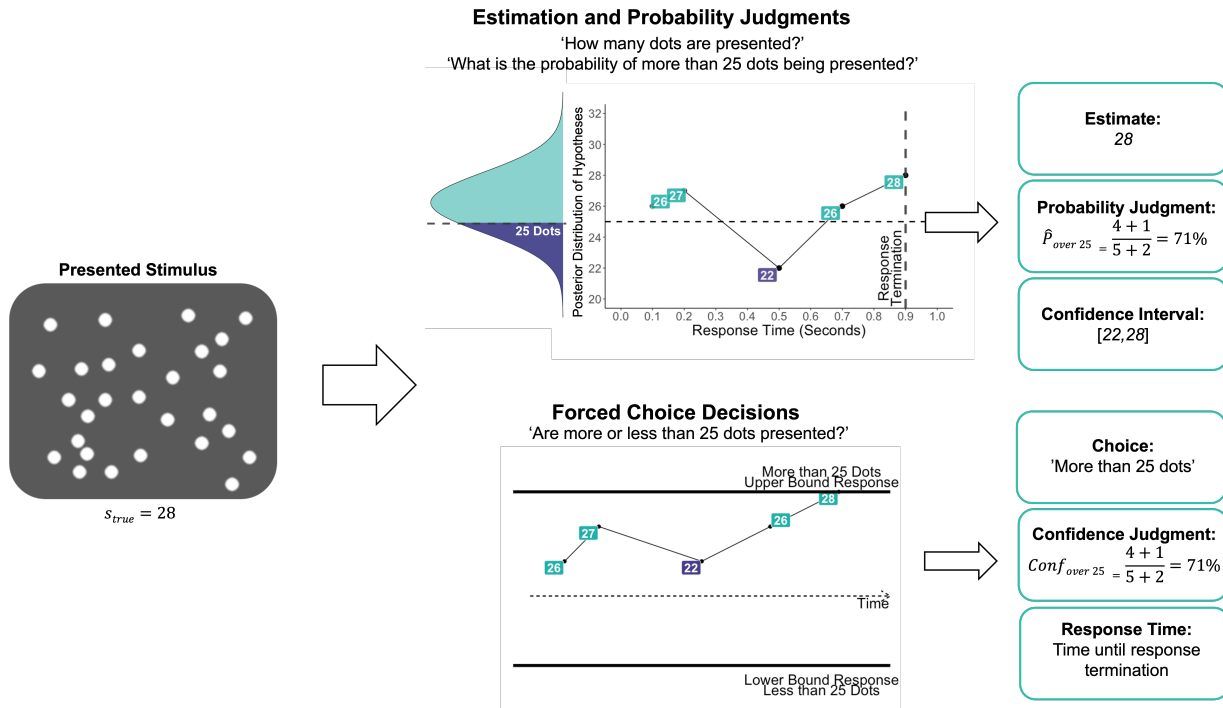


Figure 5

A Schematic illustration of the Autocorrelated Bayesian Sampler (ABS, Zhu et al., 2024) applied to a numerosity task. The stimulus (left) is mentally represented as a Gaussian distribution, from which samples are drawn to support various task responses. These include: (1) estimates, (2) probability judgments, (3) confidence intervals, (4) choices, (5) confidence judgments, and (6) response times.

Note. This figure is reprinted from "Noise in Cognition", by Sanborn et al., 2025, *Perspectives on Psychological Science*, p.12. CC-BY

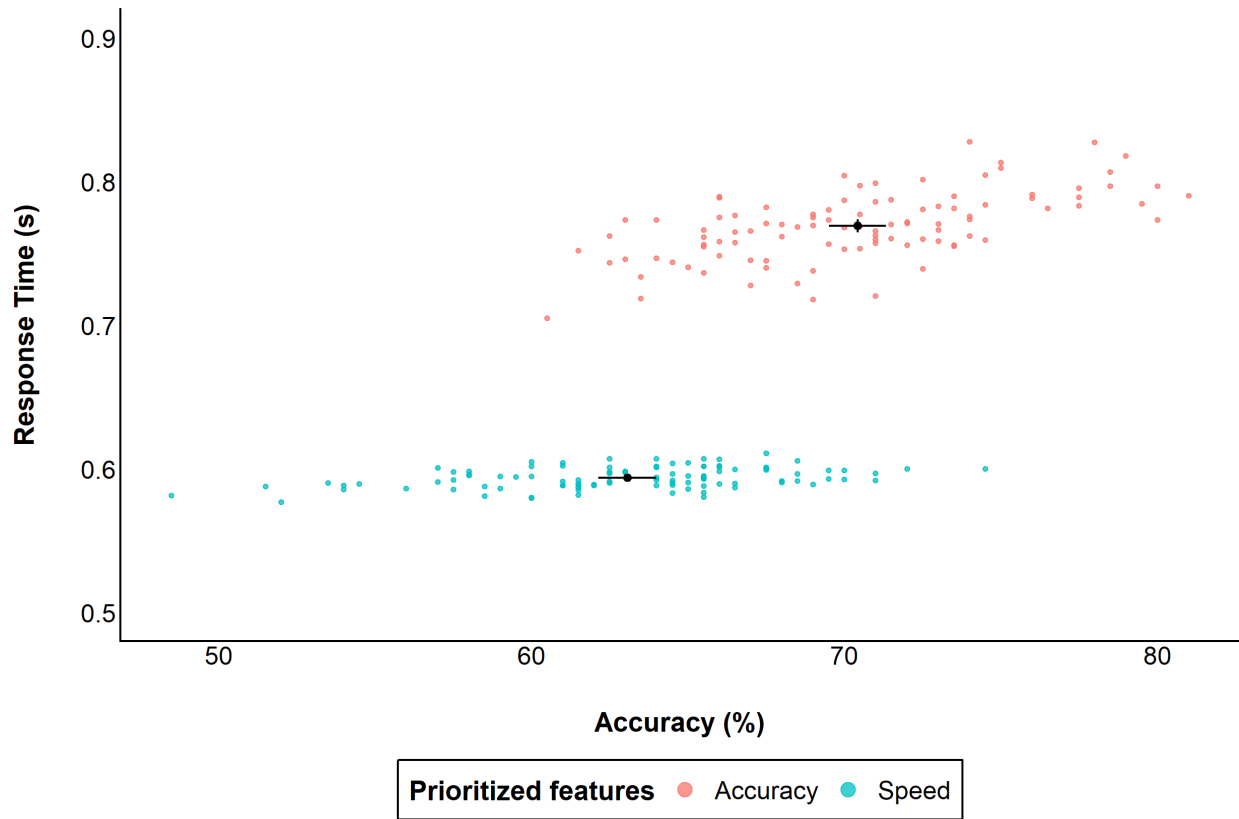


Figure 6

ABS prediction of the speed-accuracy trade-off. When the evidence threshold is small (in blue), responses are fast but exhibit reduced accuracy. Conversely, an increased evidence threshold is associated with higher accuracy and response times. The black dots mark the mean values of accuracy and response time. The black bars denote the 95% confidence intervals on both axes. ABS = Autocorrelated Bayesian Sampler.

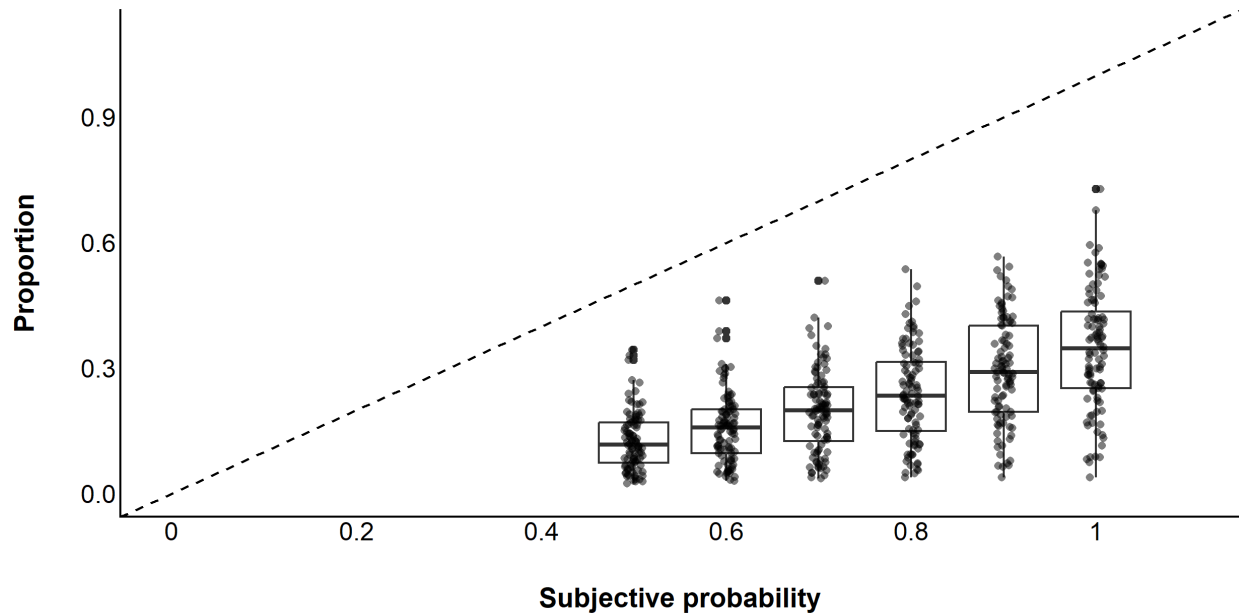


Figure 7

ABS prediction of the over-precision effect. The subjective probability represents the confidence level requested from ABS, while the proportion reflects the probability of the true distribution falling within the interval provided by ABS. The dashed line illustrates perfect calibration. ABS consistently predicts over-precise confidence intervals (dots) across all levels of subjective probability. ABS = Autocorrelated Bayesian Sampler.

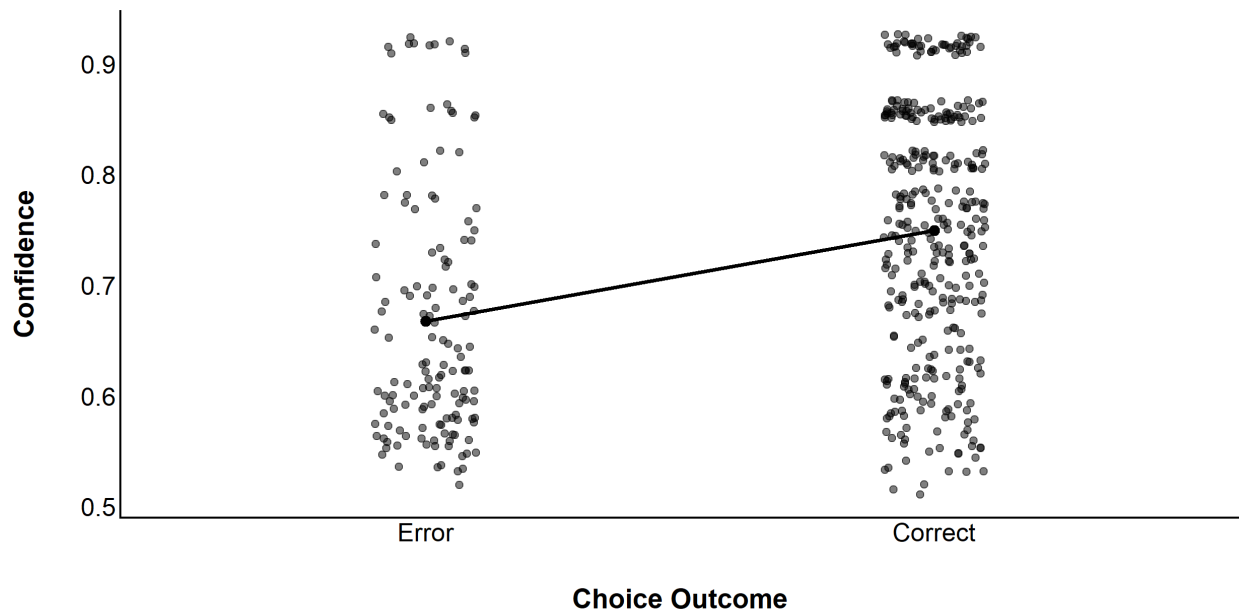


Figure 8

ABS prediction of the resolution of the confidence effect: Confidence levels are higher for correct responses than for errors. Each dot represents the confidence rating for a single trial. ABS = Autocorrelated Bayesian Sampler.

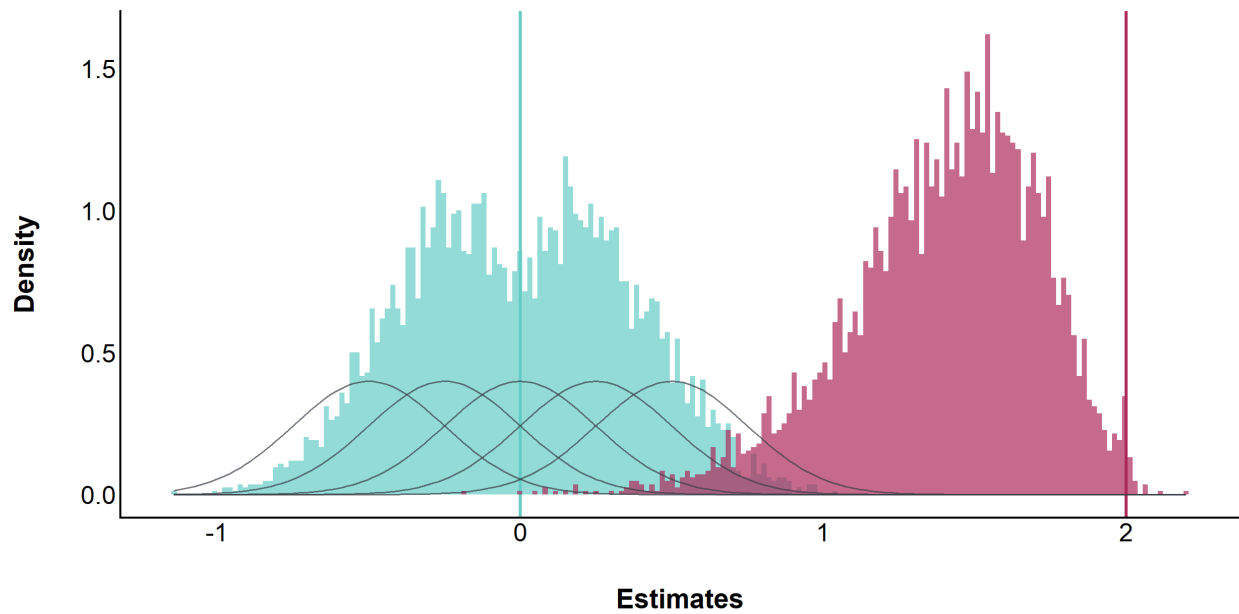


Figure 9

ABS prediction of the anchoring (in green) and repulsion effects (in red). The black bell curves represent the distribution of stimuli. Estimates are pushed away from a nearby comparison value (indicated by the green line, with the green bars showing the distribution of estimates) but pulled toward a distant comparison value (indicated by the red line, with the red bars showing the distribution of estimates). ABS = Autocorrelated Bayesian Sampler.

Model Allocation

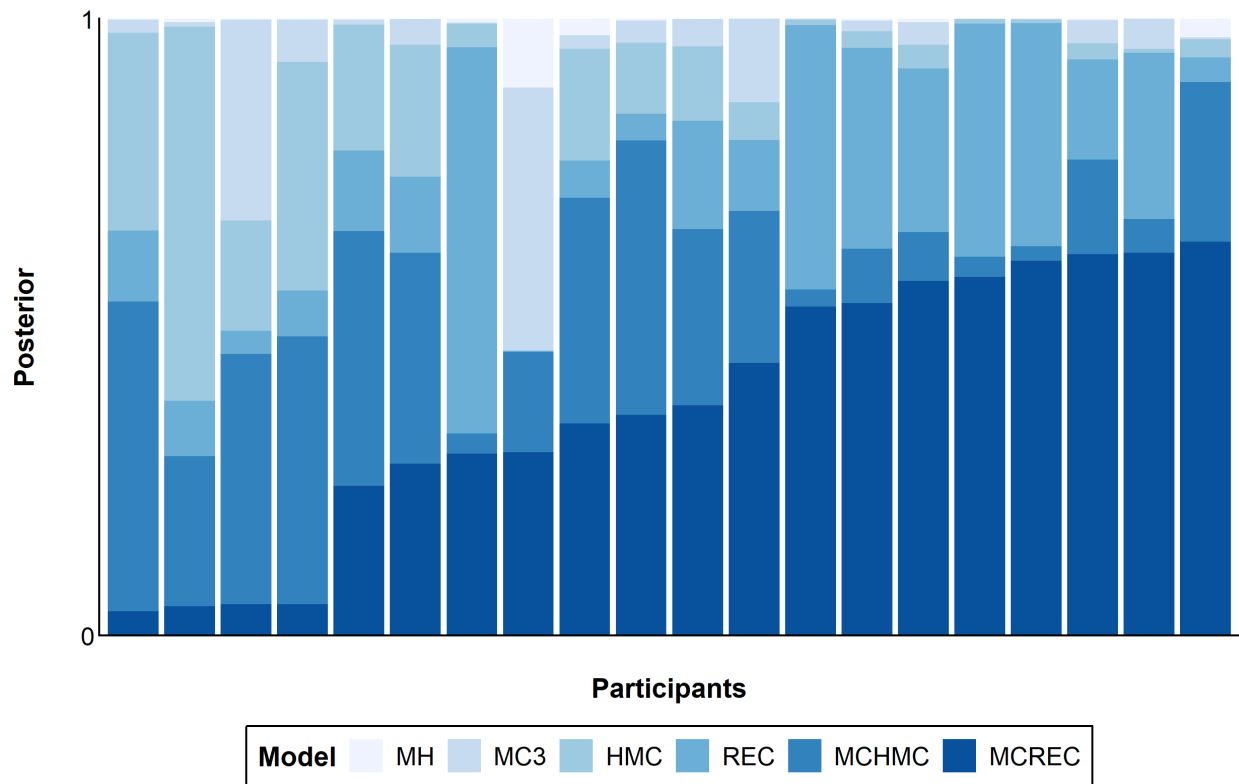


Figure 10

Model selection results for the Gaussian condition in Experiment 1 of Castillo, León-Villagr , et al. (2024), with bar lengths showing the posterior probability of models. Each column represents a different participant.

Measure	Description	Human Behaviour
Repetitions	% items equal to last	⇓
Adjacencies	% items one unit away from last	⇑
Turning Points	% of trajectory changes (asc. to desc. or vice versa)	⇓
Distances	Mean distance between consecutive items	⇓
Shape	Relative fit of Gaussian vs. Uniform densities	✓

Table 1
Random Generation Measures. Note: ⇓ too low ⇑ too high ✓ correct

Parameter Name	Value	Used by
Proposal Width	$U \sim (0, 50)$	MH, MC ³
Temperature Increase	$U \sim (1, 4)$	MC ³ , MCHMC, MCREC
Number of Chains	4, 5, 6	MC ³ , MCHMC, MCREC
Swap All	<i>yes, no</i>	MC ³ , MCHMC, MCREC
Trajectory Length	1, 2, . . . , 100	HMC, REC, MCHMC, MCREC
Recycling Rate	$U \sim (0, 1)$	REC, MCREC

Table 2
Prior over parameters for models used in quantitative model comparison example, following Castillo, León-Villagr , et al. (2024).

References

- Ariely, D., Tung Au, W., Bender, R. H., Budescu, D. V., Dietz, C. B., Gu, H., Wallsten, T. S., & Zauberman, G. (2000). The effects of averaging subjective probability estimates between and within judges. *Journal of Experimental Psychology: Applied*, 6(2), 130–147. <https://doi.org/10.1037/1076-898X.6.2.130>
- Baddeley, A. D. (1966). The Capacity for Generating Information by Randomization. *Quarterly Journal of Experimental Psychology*, 18(2), 119–129. <https://doi.org/10.1080/14640746608400019>
- Baranski, J. V., & Petrusic, W. M. (1998). Probing the locus of confidence judgments: Experiments on the time to determine confidence. *Journal of Experimental Psychology: Human Perception and Performance*, 24(3), 929–945. <https://doi.org/10.1037/0096-1523.24.3.929>
- Beaumont, M. A., Zhang, W., & Balding, D. J. (2002). Approximate Bayesian Computation in Population Genetics. *Genetics*, 162(4), 2025–2035. <https://doi.org/10.1093/genetics/162.4.2025>
- Betancourt, M. (2018). *A Conceptual Introduction to Hamiltonian Monte Carlo*. arXiv: 1701.02434. <http://arxiv.org/abs/1701.02434>
- Bhatia, S. (2017). Associative judgment and vector space semantics. *Psychological Review*, 124(1), 1–20. <https://doi.org/10.1037/rev0000047>
- Biau, G., Cérou, F., & Guyader, A. (2015). New insights into Approximate Bayesian Computation. *Annales de l'Institut Henri Poincaré, Probabilités et Statistiques*, 51(1). <https://doi.org/10.1214/13-AIHP590>
- Bramley, N. R., Dayan, P., Griffiths, T. L., & Lagnado, D. A. (2017). Formalizing Neurath's ship: Approximate algorithms for online causal learning. *Psychological Review*, 124(3), 301–338. <https://doi.org/10.1037/rev0000061>
- Brooks, S., Andrew Gelman, Jones, G., & Meng, X.-L. (Eds.). (2011). *Handbook for Markov chain Monte Carlo*. Taylor & Francis.

- Brown, S. D., & Heathcote, A. (2008). The simplest complete model of choice response time: Linear ballistic accumulation. *Cognitive Psychology*, 57(3), 153–178.
<https://doi.org/10.1016/j.cogpsych.2007.12.002>
- Bruckner, R., Nassar, M. R., Li, S.-C., & Eppinger, B. (2025). Differences in learning across the lifespan emerge via resource-rational computations. *Psychological Review*.
<https://doi.org/10.1037/rev0000526>
- Castillo, L., León-Villagr , P., Chater, N., & Sanborn, A. N. (2024). Explaining the flaws in human random generation as local sampling with momentum. *PLOS Computational Biology*, 20(1), 1–24. <https://doi.org/10.1371/journal.pcbi.1011739>
- Castillo, L., Li, Y.-X., & Sanborn, A. N. (2024, June 13). *samplrData: Datasets from the SAMPLING Project*. Comprehensive R Archive Network.
<https://doi.org/10.32614/CRAN.package.samplrData>
- Cepeda, N. J., & Munakata, Y. (2007). Why do children persevere when they seem to know better: Graded working memory, or directed inhibition? *Psychonomic Bulletin & Review*, 14(6), 1058–1065. <https://doi.org/10.3758/BF03193091>
- Chang, W. (2025). *R6: Encapsulated classes with reference semantics* (Version 2.6.1).
<https://r6.r-lib.org>
- Cooper, R. P. (2016). Executive Functions and the Generation of “Random” Sequential Responses: A Computational Account. *Journal of Mathematical Psychology*, 73, 153–168. <https://doi.org/10.1016/j.jmp.2016.06.002>
- Costello, F., & Watts, P. (2014). Surprisingly rational: Probability theory plus noise explains biases in judgment. *Psychological Review*, 121(3), 463–480.
<https://doi.org/10.1037/a0037010>
- Costello, F., & Watts, P. (2016). People’s conditional probability judgments follow probability theory (plus noise). *Cognitive Psychology*, 89, 106–133.

- Costello, F., & Watts, P. (2017). Explaining High Conjunction Fallacy Rates: The Probability Theory Plus Noise Account. *Journal of Behavioral Decision Making*, *30*(2), 304–321. <https://doi.org/10.1002/bdm.1936>
- Costello, F., & Watts, P. (2018). Invariants in probabilistic reasoning. *Cognitive Psychology*, *100*, 1–16.
- Costello, F., & Watts, P. (2019). The rationality of illusory correlation. *Psychological Review*, *126*(3), 437–450.
- Cranmer, K., Brehmer, J., & Louppe, G. (2020). The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, *117*(48), 30055–30062. <https://doi.org/10.1073/pnas.1912789117>
- Dasgupta, I., Schulz, E., & Gershman, S. J. (2017). Where do hypotheses come from? *Cognitive Psychology*, *96*, 1–25. <https://doi.org/10.1016/j.cogpsych.2017.05.001>
- Davis, Z. J., & Rehder, B. (2020). A process model of causal reasoning. *Cognitive Science*, *44*(5), e12839.
- Didelot, X., Everitt, R. G., Johansen, A. M., & Lawson, D. J. (2011). Likelihood-free estimation of model evidence. *Bayesian Analysis*, *6*(1). <https://doi.org/10.1214/11-BA602>
- Epley, N., & Gilovich, T. (2006). The Anchoring-and-Adjustment Heuristic: Why the Adjustments Are Insufficient. *Psychological Science*, *17*(4), 311–318. <https://doi.org/10.1111/j.1467-9280.2006.01704.x>
- Fearnhead, P., & Prangle, D. (2012). Constructing Summary Statistics for Approximate Bayesian Computation: Semi-Automatic Approximate Bayesian Computation. *Journal of the Royal Statistical Society Series B: Statistical Methodology*, *74*(3), 419–474. <https://doi.org/10.1111/j.1467-9868.2011.01010.x>
- Gardner, M. (1978). Mathematical games: White and brown music, fractal curves and one-over-f fluctuations. *Scientific american*, *238*(4), 16–32.

- Gershman, S. J., Vul, E., & Tenenbaum, J. B. (2012). Multistability and Perceptual Inference. *Neural Computation*, *24*(1), 1–24.
https://doi.org/10.1162/NECO_a_00226
- Geyer, C. J. (1991). Markov Chain Monte Carlo Maximum Likelihood. *Proc. 23rd Symposium on the Interface Interface Foundation, Fairfax Station, 1991*, 156–163.
- Gilden, D. L., Thornton, T., & Mallon, M. W. (1995). 1/f Noise in Human Cognition. *Science*, *267*(5205), 1837–1839. <https://www.jstor.org/stable/2886349>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). Random Forests. In *The Elements of Statistical Learning* (pp. 587–604). Springer New York.
https://doi.org/10.1007/978-0-387-84858-7_15
- Hastings, W. K. (1970). Monte Carlo sampling methods using Markov chains and their applications. *Biometrika*, *57*, 97–109.
- Hellmann, S., Zehetleitner, M., & Rausch, M. (2023). Simultaneous modeling of choice, confidence, and response time in visual perception. *Psychological Review*, *130*, 1521–1543. <https://doi.org/10.1037/rev0000411>
- Hertwig, R., Benz, B., & Krauss, S. (2008). The conjunction fallacy and the many meanings of and. *Cognition*, *108*(3), 740–753. <https://doi.org/10.1016/j.cognition.2008.06.008>
- Horowitz, A. M. (1991). A Generalized Guided Monte Carlo Algorithm. *Physics Letters B*, *268*(2), 247–252. [https://doi.org/10.1016/0370-2693\(91\)90812-5](https://doi.org/10.1016/0370-2693(91)90812-5)
- Hoyer, P., & Hyvärinen, A. (2002). Interpreting neural response variability as monte carlo sampling of the posterior. *Advances in Neural Information Processing Systems*, *15*.
- Huang, J., Busemeyer, J. R., Ebel, Z., & Pothos, E. M. (2024). Bridging the gap between subjective probability and probability judgments: The quantum sequential sampler. *Psychological Review*. <https://doi.org/10.1037/rev0000489>
- Jazayeri, M., & Movshon, J. A. (2007). A new perceptual illusion reveals mechanisms of sensory decoding. *Nature*, *446*(7138), 912–915. <https://doi.org/10.1038/nature05739>

- Juslin, P., Winman, A., & Hansson, P. (2007). The naïve intuitive statistician: A naïve sampling model of intuitive confidence intervals. *Psychological Review*, *114*(3), 678–703. <https://doi.org/10.1037/0033-295X.114.3.678>
- Juslin, P., Winman, A., & Olsson, H. (2003). Calibration, additivity, and source independence of probability judgments in general knowledge and sensory discrimination tasks. *Organizational Behavior and Human Decision Processes*, *92*(1–2), 34–51. [https://doi.org/10.1016/S0749-5978\(03\)00063-3](https://doi.org/10.1016/S0749-5978(03)00063-3)
- Kiani, R., Corthell, L., & Shadlen, M. N. (2014). Choice certainty is informed by both evidence and decision time. *Neuron*, *84*(6), 1329–1342. <https://doi.org/10.1016/j.neuron.2014.12.015>
- Kvam, P. D., Marley, A., & Heathcote, A. (2023). A unified theory of discrete and continuous responding. *Psychological Review*, *130*(2), 368. <https://doi.org/10.1037/rev0000378>
- Li, Y.-X., Falbén, J. K., Castillo, L., Spicer, J., Zhu, J.-Q., Chater, N., & Sanborn, A. N. (2024). Probability, but not utility, influences repeated mental simulations of lotteries. *Proceedings of the Annual Meeting of the Cognitive Science Society*, *46*. <https://escholarship.org/uc/item/2ps2t4sz>
- Lieder, F., Griffiths, T. L., & Hsu, M. (2018). Overrepresentation of extreme events in decision making reflects rational use of cognitive resources. *Psychological Review*, *125*(1), 1–32. <https://doi.org/10.1037/rev0000074>
- Luu, L., & Stocker, A. A. (2018). Post-decision biases reveal a self-consistency principle in perceptual inference. *eLife*, *7*, e33334. <https://doi.org/10.7554/eLife.33334>
- Macchi, L., Osherson, D., & Krantz, D. H. (1999). A note on superadditive probability judgment. *Psychological Review*, *106*(1), 210.
- Metropolis, A. W., Rosenbluth, A. W., Rosenbluth, M. N., Teller, A. H., & Teller, E. (1953). Equations of state calculations by fast computing machines. *Journal of Chemical Physics*, *21*, 1087–1092.

- Moore, D. A., & Healy, P. J. (2008). The trouble with overconfidence. *Psychological Review*, 115(2), 502–517.
- Munakata, Y., Snyder, H. R., & Chatham, C. H. (2012). Developing Cognitive Control: Three Key Transitions. *Current Directions in Psychological Science*, 21(2), 71–77. <https://doi.org/10.1177/0963721412436807>
- Neal, R. M. (1993). *Probabilistic inference using Markov Chain Monte Carlo methods* (tech. rep. No. CRG-TR-93-1). Department of Computer Science, University of Toronto.
- Neal, R. M. (2011). MCMC Using Hamiltonian Dynamics. In S. Brooks, A. Gelman, G. Jones, & X.-L. Meng (Eds.), *Handbook of Markov Chain Monte Carlo* (1st ed., pp. 113–162). Chapman and Hall/CRC. <https://doi.org/10.1201/b10905>
- Nosofsky, R. M., & Palmeri, T. J. (1997). An exemplar-based random walk model of speeded classification. *Psychological Review*, 104(2), 266–300. <https://doi.org/10.1037/0033-295X.104.2.266>
- Palestro, J. J., Sederberg, P. B., Osth, A. F., Van Zandt, T., & Turner, B. M. (2018). *Likelihood-Free Methods for Cognitive Science*. Springer International Publishing. <https://doi.org/10.1007/978-3-319-72425-6>
- Pleskac, T. J., & Busemeyer, J. R. (2010). Two-stage dynamic signal detection: A theory of choice, decision time, and confidence. *Psychological Review*, 117(3), 864–901. <https://doi.org/10.1037/a0019737>
- Pudlo, P., Marin, J.-M., Estoup, A., Cornuet, J.-M., Gautier, M., & Robert, C. P. (2016). Reliable ABC model choice via random forests. *Bioinformatics*, 32(6), 859–866. <https://doi.org/10.1093/bioinformatics/btv684>
- Rasmussen, C. E., & Ghahramani, Z. (2003). Bayesian Monte Carlo. In S. Becker, S. Thrun, & K. Obermayer (Eds.), *Advances in neural information processing systems 15* (pp. 505–512). MIT Press. <http://papers.nips.cc/paper/2150-bayesian-monte-carlo.pdf>

- Ratcliff, R. (1978). A theory of memory retrieval. *Psychological Review*, 85(2), 59–108.
<https://doi.org/10.1037/0033-295X.85.2.59>
- Ratcliff, R., & McKoon, G. (2008). The Diffusion Decision Model: Theory and Data for Two-Choice Decision Tasks. *Neural Computation*, 20(4), 873–922.
<https://doi.org/10.1162/neco.2008.12-06-420>
- Ratcliff, R., & McKoon, G. (2023). Reexamining the effects of speed–accuracy instructions with a diffusion-model-based analysis. *Journal of Experimental Psychology: Learning, Memory, and Cognition*, 49(11), 1732–1751.
<https://doi.org/10.1037/xlm0001285>
- Ratcliff, R., & Starns, J. J. (2009). Modeling confidence and response time in recognition memory. *Psychological Review*, 116(1), 59–83. <https://doi.org/10.1037/a0014086>
- Raynal, L., Marin, J.-M., Pudlo, P., Ribatet, M., Robert, C. P., & Estoup, A. (2019). ABC random forests for Bayesian parameter inference (O. Stegle, Ed.). *Bioinformatics*, 35(10), 1720–1728. <https://doi.org/10.1093/bioinformatics/bty867>
- Rhodes, T., & Turvey, M. T. (2007). Human memory retrieval as Lévy foraging. *Physica A: Statistical Mechanics and its Applications*, 385(1), 255–260.
- Ridderinkhof, K., Span, M. M., & Van Der Molen, M. W. (2002). Perseverative Behavior and Adaptive Control in Older Adults: Performance Monitoring, Rule Induction, and Set Shifting. *Brain and Cognition*, 49(3), 382–401.
<https://doi.org/10.1006/brcg.2001.1506>
- Sanborn, A. N., Yan, H., & Tsvetkov, C. (2024). Combining meta-learned models with process models of cognition. *Behavioral and Brain Sciences*, 47, e163.
<https://doi.org/10.1017/S0140525X24000165>
- Sanborn, A. N., Zhu, J.-Q., Spicer, J., Leon Villagra, P., Castillo, L., Falben, J., Li, Y.-X., Tee, A., & Chater, N. (2025). Noise in cognition: Bug or feature? *Perspectives on Psychological Science*. <https://doi.org/10.1177/17456916241258951>

- Singmann, H., Brown, S., Gretton, M., Heathcote, A., Voss, A., Voss, J., & Terry, A. (2018). *Rtdists: Response Time Distributions*. Comprehensive R Archive Network. <https://doi.org/10.32614/CRAN.package.rtdists>
- Sisson, S. A., Fan, Y., & Beaumont, M. A. (Eds.). (2019). *Handbook of approximate Bayesian computation*. CRC Press, Taylor & Francis Group.
- Smith, P. L. (2016). Diffusion theory of decision making in continuous report. *Psychological Review*, 123(4), 425–451. <https://doi.org/10.1037/rev0000023>
- Spicer, J., Zhu, J.-Q., Chater, N., & Sanborn, A. N. (2022). Perceptual and Cognitive Judgments Show Both Anchoring and Repulsion. *Psychological Science*, 33(9), 1395–1407. <https://doi.org/10.1177/09567976221089599>
- Sundh, J., Zhu, J.-Q., Chater, N., & Sanborn, A. (2023). A unified explanation of variability and bias in human probability judgments: How computational noise explains the mean–variance signature. *Journal of Experimental Psychology: General*, 152(10), 2842–2860. <https://doi.org/10.1037/xge0001414>
- Tee, A., Sundh, J., Chater, N., & Sanborn, A. N. (2024, November 14). *Modelling Probability Judgements that Violate Binary Complementarity*. <https://doi.org/10.31234/osf.io/h3rgp>
- Towse, J. N. (1998). On random generation and the central executive of working memory. *British Journal of Psychology*, 89(1), 77–101. <https://doi.org/10.1111/j.2044-8295.1998.tb02674.x>
- Turner, B. M., & Van Zandt, T. (2012). A tutorial on approximate Bayesian computation. *Journal of Mathematical Psychology*, 56(2), 69–85. <https://doi.org/10.1016/j.jmp.2012.02.005>
- Tversky, A., & Kahneman, D. (1974). Judgment under uncertainty: Heuristics and biases. *Science*, 185(4157), 1124. <https://doi.org/10.1126/science.185.4157.1124>

- Tversky, A., & Kahneman, D. (1983). Extensional versus intuitive reasoning: The conjunction fallacy in probability judgment. *Psychological Review*, 90(4), 293–315. <https://doi.org/10.1037/0033-295X.90.4.293>
- Valentin, S., Castillo, L., Sanborn, A. N., & Lucas, C. G. (2024). Distinguishing Between Process Models of Causal Learning. *Proceedings of the Annual Meeting of the Cognitive Science Society*, 46. <https://escholarship.org/uc/item/2p43j2cw>
- Van Ravenzwaaij, D., Cassey, P., & Brown, S. D. (2018). A simple introduction to Markov Chain Monte–Carlo sampling. *Psychonomic Bulletin & Review*, 25(1), 143–154. <https://doi.org/10.3758/s13423-016-1015-8>
- Vandekerckhove, J., & Tuerlinckx, F. (2008). Diffusion model analysis with MATLAB: A DMAT primer. *Behavior Research Methods*, 40(1), 61–72. <https://doi.org/10.3758/BRM.40.1.61>
- Vickers, D. (1979). *Decision processes in visual perception*. Academic Press.
- Vickers, D., & Packer, J. (1982). Effects of alternating set for speed or accuracy on response time, accuracy and confidence in a unidimensional discrimination task. *Acta Psychologica*, 50(2), 179–197. [https://doi.org/10.1016/0001-6918\(82\)90006-3](https://doi.org/10.1016/0001-6918(82)90006-3)
- Wagenaar, W. A. (1972). Generation of random sequences by human subjects: A critical survey of literature. *Psychological Bulletin*, 77, 65–72.
- Wickelgren, W. A. (1977). Speed-accuracy tradeoff and information processing dynamics. *Acta Psychologica*, 41(1), 67–85. [https://doi.org/10.1016/0001-6918\(77\)90012-9](https://doi.org/10.1016/0001-6918(77)90012-9)
- Wiecki, T. V., Sofer, I., & Frank, M. J. (2013). HDDM: Hierarchical Bayesian estimation of the Drift-Diffusion Model in Python. *Frontiers in Neuroinformatics*, 7. <https://doi.org/10.3389/fninf.2013.00014>
- Zamboni, E., Ledgeway, T., McGraw, P. V., & Schluppeck, D. (2016). Do perceptual biases emerge early or late in visual processing? Decision-biases in motion perception. *Proceedings of the Royal Society B: Biological Sciences*, 283(1833), 20160263. <https://doi.org/10.1098/rspb.2016.0263>

- 808 Zhu, J.-Q., León-Villagr , P., Chater, N., & Sanborn, A. N. (2022). Understanding the
809 structure of cognitive noise. *PLoS Computational Biology*, 18(8), e1010312.
810 <https://doi.org/10.1371/journal.pcbi.1010312>
- 811 Zhu, J.-Q., Sanborn, A. N., & Chater, N. (2020). The Bayesian sampler: Generic Bayesian
812 inference causes incoherence in human probability judgments. *Psychological Review*,
813 127(5), 719–748. <https://doi.org/10.1037/rev0000190>
- 814 Zhu, J.-Q., Sundh, J., Spicer, J., Chater, N., & Sanborn, A. N. (2024). The autocorrelated
815 Bayesian sampler: A rational process for probability judgments, estimates,
816 confidence intervals, choices, confidence judgments, and response times.
817 *Psychological Review*, 131(2), 456–493. <https://doi.org/10.1037/rev0000427>

Appendix

Implementation Details

818 Sampling Algorithms

819 The sampling algorithms offered by the `samplr` package are Markov Chain Monte
 820 Carlo algorithms (MCMC; Brooks et al., 2011). These work by initializing a Markov chain
 821 at a random point, then having the chain transition between points in a way that
 822 guarantees that, after sufficient iterations, the distribution of registered states in the chain
 823 will approximate the target distribution¹. They can be understood as stemming from a
 824 base algorithm, Metropolis-Hastings (MH; Hastings, 1970), to which qualitative
 825 modifications can be made (see Table A1). Below, we describe MH and show how other
 826 algorithms originate from modifying it.

827 *Metropolis Hastings*

828 After the Markov chain, θ , is initialized at a random point θ^1 , the
 829 Metropolis-Hastings algorithm proceeds iteratively as follows: in each iteration, it first
 830 proposes a random new location θ' by updating the current state with the addition of
 831 random normal noise (r): $\theta' \leftarrow \theta^{n-1} + r$; $r \sim N(0, \sigma^2)$, where n is the current iteration and
 832 σ , the standard deviation of the proposal distribution, is a free parameter (set by
 833 `sigma_prop` in the `samplr` functions). Then, it chooses whether to transition to θ' or not
 834 stochastically, with probability $\min\{1, \exp\{\mathcal{L}(\theta') - \mathcal{L}(\theta^{n-1})\}\}$, where $\mathcal{L}(\cdot)$ is the log
 835 probability density function of the target distribution (see Algorithm 1). Intuitively, the
 836 sampler is greedy, so that if θ' is better than θ it will always transition to this new state;
 837 but also curious, so that when θ' is worse the change is still possible.

¹ In the `samplr` package, users set the target distribution with the `distr_name` and `distr_params` parameters, if using one of the many available distributions as the target (including mixtures of these); but they can also set custom targets using the `custom_density` parameter instead. Further details on how to set target distributions are described in various vignettes in the package. Irregardless of the target distribution, the starting point must be provided by the user.

Sampling Algorithm	Function Name	Multiple Chains	Gradient-based Proposals	Autocorrelated-proposals
MH	<code>sampler_mh</code>	No	No	No
MH_{REC}^*	<code>sampler_mh</code>	No	No	Yes
HMC	<code>sampler_hmc</code>	No	Yes	No
REC	<code>sampler_rec</code>	No	Yes	Yes
MC^3	<code>sampler_mc3</code>	Yes	No	No
$\text{MC}^3_{\text{REC}}^*$	<code>sampler_mc3</code>	Yes	No	Yes
MCHMC	<code>sampler_mchmc</code>	Yes	Yes	No
MCREC	<code>sampler_mcrec</code>	Yes	Yes	Yes

Table A1

Sampling Algorithms offered by the `sampler` package, and their qualitative features. Note: Algorithms marked with an asterisk do not have their own separate function, but can be used by setting the `alpha` parameter in the corresponding function to a value other than 0

Multiple chains

One possible qualitative modification is running multiple chains which swap positions stochastically. Each chain c runs at different temperatures $\beta_c = 1/(1 + \Delta_T \times (c - 1))$, calculating the probability of a transition as $\min\{1, \exp\{(\mathcal{L}(\theta') - \mathcal{L}(\theta_c^{n-1})) \times \beta_c\}\}$, where Δ_T is the parameter (called `delta_T` in the `sampler` package) that controls how steeply the temperature increases. Then, in each iteration, after all chains have carried out both proposal and acceptance steps, a third swapping stage takes place: chains are randomly allocated to a pair, and for each pair, a position swap is proposed. These swaps are decided by comparing the joint likelihood of the current position-temperature pairs to what the joint likelihood would be if the chains swapped positions (lines 18-23 in Algorithm 2). If the swap is accepted, then the positions of the chains are exchanged. The number of chains to run is determined by a parameter C (called `nChains` in the package). Also, either all possible swaps or only one may be proposed at each iteration, which is determined by a third boolean parameter SA (called `swap_all` in the package). Although when using this modification the position of the sampler is defined by a $N \times C$ matrix θ where each row is an iteration and each column one of the chains, only the positions of the cold chain ($\theta_{c=1}$) are returned.

Algorithm 1 Metropolis Hastings

```

1: procedure METROPOLIS HASTINGS( $\theta^1, \mathcal{L}, N, \sigma$ )
2:   for n=2 to N do
3:     Sample  $r \sim N(0, \sigma^2)$ 
4:     Set  $\theta' \leftarrow \theta^{n-1} + r$  ▷ Set Proposal
5:     Set  $A \leftarrow \exp\{\mathcal{L}(\theta') - \mathcal{L}(\theta^{n-1})\}$ 
6:     Sample  $u \sim U[0, 1]$ 
7:     if  $u \leq \min\{1, A\}$  then ▷ Stochastic transition
8:       Set  $\theta^n \leftarrow \theta'$ 
9:     else
10:      Set  $\theta^n \leftarrow \theta^{n-1}$ 
11:    end if
12:  end for
13:  return  $\theta$ 
14: end procedure

```

855 *Gradient-based proposals*

856 Another modification to the Metropolis-Hastings proposal is to make proposals
857 based on the gradient of the posterior distribution by simulating a physical system.
858 Intuitively, algorithms using this modification will make proposals by simulating a
859 frictionless particle in physical space, with the topology of the space being the negative log
860 density (and thus its potential energy is proportional to that) and the kinetic momentum
861 being drawn at random from a standard normal distribution at each iteration. Once the
862 momentum has been initialized, the trajectory is simulated for a pre-set period of time, at
863 which point the ending location is recorded as the new proposal.

864 In practice, the physical simulation needs to be carried out in discrete steps, by
865 using a small stepsize ϵ and approximately computing the position at times $\epsilon, 2\epsilon, \dots, L\epsilon$ (a
866 procedure for simulating the Hamiltonian equations while minimizing error is called the
867 “leapfrog” method, which is shown in Algorithm 3). For this reason, the gradient-based
868 samplers will use a measure of how long this trajectory will be simulated for as the product
869 of $L \times \epsilon$ (these are called `L` and `epsilon` in the package) .

870 Once a proposal has been made, the transition probability is determined by the
871 joint density of the position-momentum pair (not just the density of the position), and so

Algorithm 2 Metropolis-coupled MCMC (MC³)

```

1: procedure MC3( $\theta^1, \mathcal{L}, N, \sigma, C, \Delta T, SA$ )
2:   if SA = 1 then                                ▷ Initialize how many swaps will be attempted
3:     Set  $nSwaps \leftarrow \text{floor}(C/2)$ 
4:   else
5:     Set  $nSwaps \leftarrow 1$ 
6:   end if
7:   for c=1 to C do
8:     Set  $\beta_c = 1/(1 + \Delta T \times (c - 1))$           ▷ Initialize Temperatures
9:   end for
10:  for n=2 to N do
11:    for c=1 to C do                                ▷ Each chain samples independently
12:      Sample  $r \sim N(0, \sigma^2)$ 
13:      Set  $\theta' \leftarrow \theta_c^{n-1} + r$ 
14:      Set  $A \leftarrow \exp\{(\mathcal{L}(\theta') - \mathcal{L}(\theta_c^{n-1})) \times \beta_c\}$     ▷ A softened by the temperature
15:      Sample  $u \sim U[0, 1]$ 
16:      if  $u \leq \min\{1, A\}$  then Set  $\theta_c^n \leftarrow \theta'$  else Set  $\theta_c^n \leftarrow \theta_c^{n-1}$  end if
17:    end for
18:    for s=1 to nSwaps do                            ▷ Propose swaps after all chains have made a step
19:      Randomly select two chains  $i, j$  without repetition
20:      Sample  $u \sim U[0, 1]$ 
21:      Set  $A_{\text{swap}} \leftarrow \exp\{(\mathcal{L}(\theta_j^n) \times \beta_i) + (\mathcal{L}(\theta_i^n) \times \beta_j) - (\mathcal{L}(\theta_i^n) \times \beta_i) - (\mathcal{L}(\theta_j^n) \times \beta_j)\}$ 
22:      if  $u \leq \min\{1, A\}$  then Swap  $\theta_i^n$  with  $\theta_j^n$  end if
23:    end for
24:  end for
25:  return  $\theta_{c=1}$                                 ▷ Only consider the cold chain
26: end procedure

```

872 the proposal will be accepted with probability $\min\{1, \frac{\exp\{\mathcal{L}(\theta') - \frac{1}{2}p' \cdot p'\}}{\exp\{\mathcal{L}(\theta^{n-1}) - \frac{1}{2}p \cdot p\}}\}$. Notice that the last
873 step of the leapfrog algorithm (line 8) is to negate the momentum p' , which makes the
874 Markov chain reversible.

875 If the sampler ran multiple chains, the swapping procedure in Algorithm 2, lines
876 17-22 would follow. Note that the joint density, not the density alone, should be used to
877 calculate whether a swap should occur, and that the momentum variable for the i^{th} chain
878 should be drawn from $p \sim N(0, 1/\beta_i)$, not $p \sim N(0, 1)$.

879 For more information on gradient-based proposals and the leapfrog method, see
880 Neal (2011) and Betancourt (2018).

Algorithm 3 Hamiltonian Monte Carlo

```

1: procedure LEAPFROG( $\theta', p', \mathcal{L}, \epsilon, L$ )
2:    $p' \leftarrow p' - \epsilon/2 \times \nabla \mathcal{L}(\theta')$  ▷ Start with half step
3:   for  $i=1$  to  $L$  do
4:      $\theta' \leftarrow \theta' + \epsilon \times p'$ 
5:     if  $i \neq L$  then  $p' \leftarrow p' - \epsilon \times \nabla \mathcal{L}(\theta')$  end if
6:   end for
7:    $p' \leftarrow p' - \epsilon/2 \times \nabla \mathcal{L}(\theta')$  ▷ End with half step instead of full step
8:    $p' \leftarrow -1 \times p'$  ▷ Negate momentum to keep proposal symmetric
9:   return  $\theta', p'$ 
10: end procedure
11:
12: procedure HMC( $\theta^0, \mathcal{L}, N, \epsilon, L$ )
13:   for  $n=1$  to  $N$  do
14:      $\theta' \leftarrow \theta^{n-1}$ 
15:      $p \sim N(0, 1); p' \leftarrow p$  ▷ Initialize Momentum and make a copy
16:      $\theta', p' \leftarrow \text{Leapfrog}(\theta', p', \mathcal{L}, \epsilon, L)$  ▷ Proposal simulating physical system
17:      $A \leftarrow \min\{1, \frac{\exp\{\mathcal{L}(\theta') - \frac{1}{2}p' \cdot p'\}}{\exp\{\mathcal{L}(\theta^{n-1}) - \frac{1}{2}p \cdot p\}}\}$ 
18:      $u \sim U[0, 1]$ 
19:     if  $u \leq A$  then
20:        $\theta^n \leftarrow \theta'$ 
21:     else
22:        $\theta^n \leftarrow \theta^{n-1}$ 
23:     end if
24:   end for
25:   return  $\theta$ 
26: end procedure

```

Autocorrelated Proposals

The final possible modification implemented is to use autocorrelated proposals, as proposed by Horowitz (1991). A sampler using this approach uses one additional parameter, α , (called **alpha** in the **samplr** functions); $\alpha \in U[-1, 1]$, with negative values leading to anticorrelated behavior and $\alpha = 0$ being uncorrelated behavior.

Algorithms with this modification and gradient-based proposals make a slight modification to how the momentum is drawn (ln. 15 in Algorithm 3), so that

$$p \leftarrow \alpha \times p^{n-1} + (1 - \alpha^2)^{\frac{1}{2}} \times v; \quad v \sim N(0, 1) \quad (\text{A1})$$

where p^{n-1} is the momentum at the end of the last iteration (p'). Because in the standard leapfrog algorithm the momentum is negated at the end, samplers with this modification will have to negate the momentum again (to undo the change of direction that changing the sign involves). In the first iteration of the sampler and after a chain has swapped position (in samplers with multiple chains) the momentum is not recycled but drawn from a standard normal distribution (as in HMC).

For sampling algorithms that do not use gradient-based proposals, this modification applies to how proposals are produced. While typically a proposed item is generated by adding normal unbiased noise r to θ^{n-1} (line 4 in Algorithm 1), here a history of added noise is kept, and the noise added to θ^{n-1} is a vector r , produced following Equation A1, substituting the following terms in this Equation: $v \sim N(0, 1)$ with $v \sim N(0, \sigma)$ and p and p^{n-1} with r and r^{n-1} ; with r^{n-1} being the noise that was added to θ^{n-2} in the previous iteration. In the first iteration of the sampler and after a chain has swapped position (in samplers with multiple chains) r is drawn from $N(0, \sigma^2)$.

Cognitive Models

The `samplr` package supports simulation operations for two cognitive models: the Bayesian Sampler (BS; Zhu et al., 2020) and the Autocorrelated Bayesian Sampler (ABS; Zhu et al., 2024). In the sections below, we describe how BS generates probability judgments, and how ABS produces estimates, probability judgments, confidence intervals, choices, confidence judgments, and response times within a single trial.

Bayesian Sampler

According to the BS, the mean and variance of a probability judgment can be analytically derived from the true probability p of the target event (i.e., the conjunctions and disjunctions of A and B in the `Bayesian_Sampler` function), the fixed sample size parameter N (the argument `N`) and the Beta prior β (the argument `beta`). To simulate a probability judgment, the model first draws a random value X from a Binomial distribution to represent the number of times the event occurs out of N samples. The final

Algorithm 4 Bayesian Sampler

```

1: procedure BAYESIAN_SAMPLER( $p, N, \beta$ )
2:    $M \leftarrow \frac{pN}{N + 2\beta} + \frac{\beta}{N + 2\beta}$   $\triangleright$  The mean of the probability judgment
3:    $V \leftarrow \frac{Np(1-p)}{(N + 2\beta)^2}$   $\triangleright$  The variance of the probability judgment
4:   Sample  $X \sim \text{Binomial}(N, p)$   $\triangleright$  The simulated probability judgment for one trial
5:    $P \leftarrow \frac{X + \beta}{N + 2\beta}$ 
6:   return  $M, V, P$ 
7: end procedure

```

915 probability judgment is then computed according to the BS formula, incorporating X , N
916 and the symmetric Beta prior β .

917 ***Autocorrelated Bayesian Sampler***

918 ABS implements two distinct stopping rules: for probability judgments, point
919 estimates, and confidence intervals, it uses a fixed stopping rule, whereas for decisions
920 involving two alternatives, it applies a relative stopping rule.

921 As shown in Algorithm 5, under the fixed stopping rule (specified via
922 `stopping_rule = 'fixed'` in `Zhu23ABS()`), ABS draws a fixed number of samples δ (set
923 by `n_samples`) using the MC³ sampler. Because the MC³ sampler requires a starting point
924 θ^1 (called `start_point`), ABS will draw an initial value from a Gaussian distribution
925 $N(\text{Stim}, \sigma_d^2)$ if the user does not provide one. Here, `Stim` refers to the stimulus value (set
926 via `stimulus`), and σ_d is the standard deviation (given by `distr_params`).

927 The total decision time RT is computed as the sum of a non-decision time t_0 and a
928 sampling time t_s . The non-decision time is drawn from a uniform distribution, while the
929 sampling time follows an Erlang distribution. The point estimate PE is defined as the last
930 sample in the sequence. The lower and upper bounds of a confidence interval at a given
931 level α (given by `conf_level`) are computed from the quantiles of the sample sequence.

932 Under the relative stopping rule (specified via `stopping_rule = 'relative'` in
933 `Zhu23ABS()`), ABS operates differently from the fixed stopping rule (see Algorithm 6). The

process begins by generating a starting point θ^1 if the user does not provide one. However, in the relative stopping rule, the target distribution is defined as $N(d'(\text{Stim} - \frac{1}{2}), \sigma_d^2)$, where d' represents the discriminability of the two alternatives (called `discrim` in `Zhu23ABS()`). The stimulus `Stim` must be either 0 or 1, indicating the two alternatives. Thus when `Stim` = 0, the target distribution becomes $N(-\frac{d'}{2}, \sigma_d^2)$; while when `Stim` = 1, the target distribution is $N(\frac{d'}{2}, \sigma_d^2)$.

ABS also incorporates a Beta prior, $\text{Beta}(\beta_1, \beta_2)$ (defined by `prior_on_resp` in `Zhu23ABS`), which reflects the prior on responses before the evidence accumulation. Lines 5–9 of Algorithm 6 describe the mechanism of adaptive priors for repeated choices, which can be enabled or disabled using the `prior_depend` argument in `Zhu23ABS`. This mechanism allows the model to incorporate historical information into the prior. If there is no adaptive prior, the prior on responses for all the trials will be: $\beta'_1 \leftarrow \beta_1$, $\beta'_2 \leftarrow \beta_2$. Lines 10–35 in Algorithm 6 mirror the steps of the MC^3 algorithm, with the key difference that each sampled value will be compared against the decision boundary B (specified by `dec_bdry`). Evidence is accumulated based on the side of the boundary on which each sample falls. The relative difference between the accumulated evidence is then compared to the threshold δ (called `delta`), which determines when sampling stops.

Lines 37–40 in Algorithm 6 demonstrate how ABS generates responses, determines accuracy, and computes decision confidence. The remaining steps are identical to those in the fixed stopping rule implementation.

Algorithm 5 Autocorrelated Bayesian Sampler with fixed stopping rule for one trial

```

1: procedure ZHUABS( $\sigma, C, T_0, ST_0, \sigma_d, \lambda, \delta, \alpha, \text{Stim}, \theta^1$ )
2:   if  $\theta^1$  is NA then ▷ If the starting point is not provided
3:     Draw  $\theta^1 \sim N(\text{Stim}, \sigma_d^2)$ 
4:   end if
5:    $\theta \leftarrow MC^3(\theta^1, \mathcal{L}, \delta, C, \Delta T = 4, SA = 1)$  ▷  $\mathcal{L}$  is the log density function of the
   Gaussian distribution  $N(\text{Stim}, \sigma_d^2)$ 
6:   Sample  $t_0 \sim U[T_0, T_0 + ST_0]$  ▷ The non-decision time; when  $ST_0 = 0$ ,  $t_0 = T_0$ 
7:   Sample  $t_s \sim \text{Erlang}(\delta, \lambda)$  ▷ The sampling time
8:    $RT = t_0 + t_s$  ▷ The decision time is the sum of the non-decision and sampling time
9:    $PE = \theta^\delta$  ▷ The point estimate is the last sample of the sequence
10:   $LC = \text{Quantile}(\theta, \frac{1-\alpha}{2})$  ▷  $LC$  is the  $\frac{1-\alpha}{2}$  quantiles of the samples  $\theta$ 
11:   $UC = \text{Quantile}(\theta, \frac{1+\alpha}{2})$ 
12:  return  $\theta, \text{Stim}, RT, PE, LC, UC$ 
13: end procedure

```

Algorithm 6 Autocorrelated Bayesian Sampler with relative stopping rule for one trial

Require: $\text{Stim} \in \{0, 1\}$

```

1: procedure ZHUABS( $\sigma, C, T_0, ST_0, \sigma_d, \lambda, \delta, d', B, \beta_1, \beta_2, \alpha, \text{Stim}, \theta^1, N_{max}$ )
2:   if  $\theta^1$  is NA then ▷ If the starting point is not provided
3:     Draw  $\theta^1 \sim N(d'(\text{Stim} - \frac{1}{2}), \sigma_d^2)$  ▷ In multiple trials,  $\theta^1$  will be the last sample of
     the previous trial
4:   end if
5:   if  $\text{Stim}_{\text{prev\_trial}} = 0$  then ▷ If the stimuli of the previous trial is 0
6:      $\beta'_1 \leftarrow \beta_1 + 1, \beta'_2 \leftarrow \beta_2$ 
7:   else
8:      $\beta'_1 \leftarrow \beta_1, \beta'_2 \leftarrow \beta_2 + 1$ 
9:   end if
10:  Set  $nSwaps \leftarrow \text{floor}(C/2)$  ▷ Lines 10-35 mirror the MC3 program except that each
   sample will be evaluated against the decision boundary
11:  for  $c=1$  to  $C$  do
12:     $\Delta T = 4$ 
13:    Set  $\beta_c = 1/(1 + \Delta T \times (c - 1))$ 
14:  end for
15:  for  $n=2$  to  $N_{max}$  do
16:    for  $c=1$  to  $C$  do
17:      Sample  $r \sim N(0, \sigma^2)$ 
18:      Set  $\theta' \leftarrow \theta_c^{n-1} + r$ 
19:      Set  $A \leftarrow \exp\{(\mathcal{L}(\theta') - \mathcal{L}(\theta_c^{n-1})) \times \beta_c\}$  ▷  $\mathcal{L}$  is the log density function of
   Gaussian distribution  $N(d'(\text{Stim} - \frac{1}{2}), \sigma_d^2)$ 
20:      Sample  $u \sim U[0, 1]$ 
21:      if  $u \leq \min\{1, A\}$  then Set  $\theta_c^n \leftarrow \theta'$  else Set  $\theta_c^n \leftarrow \theta_c^{n-1}$  end if
22:    end for

```

Algorithm 6 Autocorrelated Bayesian Sampler with relative stopping rule for one trial (continued)

```

23:   for s=1 to nSwaps do
24:       Randomly select two chains  $i, j$  without repetition
25:       Sample  $u \sim U[0, 1]$ 
26:       Set  $A_{swap} \leftarrow \exp\{(\mathcal{L}(\theta_j^n) \times \beta_i) + (\mathcal{L}(\theta_i^n) \times \beta_j) - (\mathcal{L}(\theta_i^n) \times \beta_i) - (\mathcal{L}(\theta_j^n) \times \beta_j)\}$ 
27:       if  $u \leq \min\{1, A\}$  then Swap  $\theta_i^n$  with  $\theta_j^n$  end if
28:   end for
29:   if  $\theta_{c=1}^n < B$  then ▷ Evaluate each sample with the decision boundary and
       accumulate the evidence
30:        $\beta'_1 \leftarrow \beta'_1 + 1$ 
31:   else
32:        $\beta'_2 \leftarrow \beta'_2 + 1$ 
33:   end if
34:   if  $|\beta'_1 - \beta'_2| = \delta$  then BREAK end if ▷ The relative stopping rule
35: end for
36:  $RS \leftarrow \text{ConvertBooleanToNumeric}(\beta'_1 < \beta'_2)$  ▷ Convert TRUE to 1 and FALSE to 0
37:  $AC \leftarrow \text{ConvertBooleanToNumeric}(RS = \text{Stim})$ 
38:  $CF = \max\left\{\frac{\beta'_1}{\beta'_1 + \beta'_2}, \frac{\beta'_2}{\beta'_1 + \beta'_2}\right\}$  ▷ The confidence of the response
39: Sample  $t_0 \sim U[T_0, T_0 + ST_0]$  ▷ The non-decision time; when  $ST_0 = 0$ ,  $t_0 = T_0$ 
40: Sample  $t_s \sim \text{Erlang}(\delta, \lambda)$ 
41:  $RT = t_0 + t_s$ 
42:  $PE = \theta_{c=1}^n$  ▷ The point estimate is the last sample of the cold chain
43:  $LC = \text{Quantile}(\theta_{c=1}, \frac{1-\alpha}{2})$ 
44:  $UC = \text{Quantile}(\theta_{c=1}, \frac{1+\alpha}{2})$ 
45: return  $\theta_{c=1}, RS, \text{Stim}, AC, RT, PE, LC, UC$ 
46: end procedure

```
